



# Deep Packet Inspection Engine

## Comprehensive Technical Documentation

User Traffic (PCAP) → [DPI Engine] → Filtered Traffic (PCAP)

**Project:** Packet Analyzer

**Language:** C++17

**Components:** PCAP Reader, Protocol Parser, SNI Extractor,  
Connection Tracker, Load Balancer, Fast Path Processor, DPI Engine

# Contents

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                     | <b>4</b>  |
| 1.1      | Purpose of This Document . . . . .                      | 4         |
| 1.2      | Project Summary . . . . .                               | 4         |
| 1.3      | Real-World Applications . . . . .                       | 4         |
| <b>2</b> | <b>Networking Fundamentals</b>                          | <b>5</b>  |
| 2.1      | The OSI Model and Network Layering . . . . .            | 5         |
| 2.2      | The Ethernet Frame (Layer 2) . . . . .                  | 6         |
| 2.2.1    | Ethernet Parsing in Source Code . . . . .               | 6         |
| 2.3      | The IPv4 Datagram (Layer 3) . . . . .                   | 7         |
| 2.3.1    | Key Fields . . . . .                                    | 7         |
| 2.3.2    | IPv4 Header in Source Code . . . . .                    | 7         |
| 2.4      | TCP — Transmission Control Protocol (Layer 4) . . . . . | 8         |
| 2.4.1    | TCP Header Structure . . . . .                          | 8         |
| 2.4.2    | TCP Flags . . . . .                                     | 8         |
| 2.4.3    | The TCP Three-Way Handshake . . . . .                   | 9         |
| 2.5      | UDP — User Datagram Protocol (Layer 4) . . . . .        | 9         |
| 2.6      | The Five-Tuple: Identifying a Network Flow . . . . .    | 9         |
| 2.7      | The PCAP File Format . . . . .                          | 10        |
| 2.7.1    | Global Header (24 bytes) . . . . .                      | 10        |
| 2.7.2    | Per-Packet Header (16 bytes) . . . . .                  | 11        |
| 2.7.3    | PCAP Structures in Source Code . . . . .                | 11        |
| <b>3</b> | <b>Deep Packet Inspection — Theory</b>                  | <b>12</b> |
| 3.1      | What is Deep Packet Inspection? . . . . .               | 12        |
| 3.2      | The HTTPS Paradox . . . . .                             | 12        |
| 3.3      | The TLS Handshake . . . . .                             | 13        |
| 3.4      | TLS Client Hello Structure . . . . .                    | 13        |
| 3.4.1    | SNI Extension Binary Format . . . . .                   | 14        |
| 3.5      | HTTP Host Header (Unencrypted Traffic) . . . . .        | 14        |
| 3.6      | DNS Query Extraction . . . . .                          | 14        |
| 3.7      | QUIC Protocol . . . . .                                 | 15        |
| 3.8      | Application Classification . . . . .                    | 15        |
| 3.9      | Connection States . . . . .                             | 15        |
| 3.10     | Packet Actions . . . . .                                | 16        |
| <b>4</b> | <b>Project Architecture</b>                             | <b>16</b> |
| 4.1      | Two-Mode Design . . . . .                               | 16        |
| 4.1.1    | Simple Single-Threaded Mode . . . . .                   | 16        |
| 4.1.2    | Multi-Threaded DPI Mode . . . . .                       | 16        |
| 4.2      | Thread Roles . . . . .                                  | 17        |
| 4.3      | Why Consistent Hashing? . . . . .                       | 17        |
| 4.4      | Thread-Safe Queue . . . . .                             | 18        |
| <b>5</b> | <b>Component Deep-Dives</b>                             | <b>18</b> |
| 5.1      | PcapReader — <code>src/pcap_reader.cpp</code> . . . . . | 19        |
| 5.1.1    | Responsibility . . . . .                                | 19        |

|       |                                                                        |    |
|-------|------------------------------------------------------------------------|----|
| 5.1.2 | Open and Validate . . . . .                                            | 19 |
| 5.1.3 | Reading Packets . . . . .                                              | 19 |
| 5.2   | PacketParser — <code>src/packet_parser.cpp</code> . . . . .            | 20 |
| 5.2.1 | Responsibility . . . . .                                               | 20 |
| 5.2.2 | Top-Level Parse Method . . . . .                                       | 20 |
| 5.2.3 | Ethernet Parsing . . . . .                                             | 20 |
| 5.2.4 | IPv4 Parsing . . . . .                                                 | 21 |
| 5.2.5 | TCP Parsing . . . . .                                                  | 21 |
| 5.2.6 | Helper: IP Address to String . . . . .                                 | 22 |
| 5.3   | SNI Extractor — <code>src/sni_extractor.cpp</code> . . . . .           | 22 |
| 5.3.1 | Responsibility . . . . .                                               | 22 |
| 5.3.2 | Detection: Is This a TLS Client Hello? . . . . .                       | 22 |
| 5.3.3 | Extraction: Navigating to the SNI . . . . .                            | 23 |
| 5.3.4 | HTTP Host Extraction . . . . .                                         | 24 |
| 5.4   | Rule Manager — <code>src/rule_manager.cpp</code> . . . . .             | 24 |
| 5.4.1 | Responsibility . . . . .                                               | 24 |
| 5.4.2 | Thread Safety Design . . . . .                                         | 24 |
| 5.4.3 | Domain Wildcard Matching . . . . .                                     | 25 |
| 5.4.4 | Combined Rule Check . . . . .                                          | 25 |
| 5.4.5 | Rule Persistence . . . . .                                             | 26 |
| 5.5   | Connection Tracker — <code>src/connection_tracker.cpp</code> . . . . . | 26 |
| 5.5.1 | Responsibility . . . . .                                               | 26 |
| 5.5.2 | The Connection Hash Map . . . . .                                      | 26 |
| 5.5.3 | Get or Create . . . . .                                                | 26 |
| 5.5.4 | Bidirectional Flow Lookup . . . . .                                    | 27 |
| 5.5.5 | Stale Connection Cleanup . . . . .                                     | 27 |
| 5.5.6 | Per-Flow Statistics . . . . .                                          | 28 |
| 5.5.7 | Global Connection Table . . . . .                                      | 28 |
| 5.6   | Load Balancer — <code>src/load_balancer.cpp</code> . . . . .           | 29 |
| 5.6.1 | Responsibility . . . . .                                               | 29 |
| 5.6.2 | Main Processing Loop . . . . .                                         | 29 |
| 5.6.3 | LBManager: Routing the Reader to LBs . . . . .                         | 29 |
| 5.7   | Fast Path Processor — <code>src/fast_path.cpp</code> . . . . .         | 30 |
| 5.7.1 | Responsibility . . . . .                                               | 30 |
| 5.7.2 | Main Loop and Processing . . . . .                                     | 30 |
| 5.7.3 | Payload Inspection Logic . . . . .                                     | 31 |
| 5.7.4 | TCP State Tracking . . . . .                                           | 31 |
| 5.7.5 | Rule Checking . . . . .                                                | 32 |
| 5.8   | DPI Engine — <code>src/dpi_engine.cpp</code> . . . . .                 | 32 |
| 5.8.1 | Responsibility . . . . .                                               | 32 |
| 5.8.2 | Initialization Sequence . . . . .                                      | 32 |
| 5.8.3 | Startup and Shutdown Order . . . . .                                   | 33 |
| 5.8.4 | Reader Thread . . . . .                                                | 33 |
| 5.8.5 | Output Handling . . . . .                                              | 34 |
| 5.8.6 | Statistics and Reporting . . . . .                                     | 34 |
| 6     | Data Flow: Tracing One Packet End-to-End . . . . .                     | 35 |
| 7     | Build System and Compilation . . . . .                                 | 35 |

|           |                                                                     |           |
|-----------|---------------------------------------------------------------------|-----------|
| 7.1       | CMake Configuration . . . . .                                       | 35        |
| 7.2       | Compiler Requirements . . . . .                                     | 36        |
| 7.3       | Generating Test Traffic . . . . .                                   | 36        |
| <b>8</b>  | <b>Running the Engine and Interpreting Output</b>                   | <b>36</b> |
| 8.1       | Simple Mode . . . . .                                               | 36        |
| 8.2       | DPI Multi-Threaded Mode . . . . .                                   | 37        |
| 8.3       | Rule File Management . . . . .                                      | 37        |
| <b>9</b>  | <b>Web Deployment Architecture</b>                                  | <b>38</b> |
| 9.1       | Deployment Overview . . . . .                                       | 38        |
| 9.2       | Monorepo Folder Structure . . . . .                                 | 39        |
| 9.3       | render.yaml — Infrastructure as Code . . . . .                      | 39        |
| 9.4       | FastAPI Backend — <code>backend-render/server.py</code> . . . . .   | 40        |
| 9.4.1     | Responsibility . . . . .                                            | 40        |
| 9.4.2     | CORS Configuration . . . . .                                        | 40        |
| 9.4.3     | File Upload and Binary Invocation . . . . .                         | 40        |
| 9.4.4     | API Endpoints . . . . .                                             | 41        |
| 9.5       | main_json.cpp — Web-Optimised C++ Entry Point . . . . .             | 41        |
| 9.5.1     | Responsibility . . . . .                                            | 41        |
| 9.5.2     | JSON Output Schema . . . . .                                        | 42        |
| 9.5.3     | Application Detection Logic in main_json.cpp . . . . .              | 42        |
| 9.5.4     | JSON String Escaping . . . . .                                      | 43        |
| 9.6       | React Frontend — <code>frontend-vercel/src/App.jsx</code> . . . . . | 43        |
| 9.6.1     | Responsibility . . . . .                                            | 43        |
| 9.6.2     | Component Architecture . . . . .                                    | 43        |
| 9.6.3     | Four Dashboard Tabs . . . . .                                       | 44        |
| 9.6.4     | File Upload and Analysis Flow . . . . .                             | 44        |
| 9.6.5     | Vite Configuration and Vercel Deployment . . . . .                  | 44        |
| 9.7       | Docker Multi-Stage Build . . . . .                                  | 45        |
| 9.8       | End-to-End Web Request Lifecycle . . . . .                          | 46        |
| 9.9       | Security Considerations in Web Deployment . . . . .                 | 47        |
| <b>10</b> | <b>Comparison: CLI vs. Web Deployment Modes</b>                     | <b>47</b> |
| <b>11</b> | <b>Performance Considerations</b>                                   | <b>47</b> |
| 11.1      | Zero-Copy Payload Inspection . . . . .                              | 47        |
| 11.2      | Lock-Free Counters . . . . .                                        | 48        |
| 11.3      | Per-Thread Connection Tables . . . . .                              | 48        |
| 11.4      | Bounded Queues and Backpressure . . . . .                           | 48        |
| 11.5      | Scalability . . . . .                                               | 48        |
| <b>12</b> | <b>Limitations and Future Enhancements</b>                          | <b>48</b> |
| 12.1      | Current Limitations . . . . .                                       | 48        |
| 12.2      | Potential Enhancements . . . . .                                    | 49        |
| <b>13</b> | <b>Glossary</b>                                                     | <b>49</b> |
| <b>14</b> | <b>Conclusion</b>                                                   | <b>51</b> |

# 1 Introduction

---

## 1.1 Purpose of This Document

This document provides a comprehensive technical explanation of the **DPI Engine** (Deep Packet Inspection Engine), a C++17 multi-threaded network packet analysis system.[cite: 1] It is intended for readers who wish to understand both the *networking theory* underpinning the project and the *software architecture* that implements it.[cite: 1]

By the end of this document, the reader should be able to:[cite: 1]

- Understand how network packets are structured at each protocol layer.[cite: 1]
- Explain what Deep Packet Inspection is and why it is significant.[cite: 1]
- Describe how TLS Server Name Indication (SNI) enables application identification.[cite: 1]
- Trace the full lifecycle of a packet through the engine.[cite: 1]
- Understand the multi-threaded pipeline architecture and its design rationale.[cite: 1]
- Read and interpret the source code of each component.[cite: 1]

## 1.2 Project Summary

The DPI Engine reads network traffic captures (.pcap files), parses each packet through the Ethernet / IP / TCP / UDP protocol stack, inspects application-layer payloads to identify applications and websites, applies configurable blocking rules, and writes the permitted traffic to an output PCAP file.[cite: 1]

**Key Capability:** Even for HTTPS (encrypted) traffic, the engine can identify the destination website by reading the *Server Name Indication* field in the TLS Client Hello — a field that is transmitted in **plaintext** before the encrypted session begins.[cite: 1]

The project provides two operational modes:[cite: 1]

1. **Simple / Single-threaded mode** (src/main\_working.cpp) — for learning and small captures.[cite: 1]
2. **Multi-threaded DPI mode** (src/dpi\_mt.cpp) — a production-grade pipeline with dedicated Load Balancer and Fast Path Processor threads.[cite: 1]

## 1.3 Real-World Applications

Deep Packet Inspection is used in many critical systems:[cite: 1]

| Application                   | Description                                                             |
|-------------------------------|-------------------------------------------------------------------------|
| <b>ISP Traffic Management</b> | Throttle bandwidth-heavy applications such as video streaming.[cite: 1] |
| <b>Enterprise Firewalls</b>   | Block social media or gaming platforms on office networks.[cite: 1]     |
| <b>Parental Controls</b>      | Prevent access to inappropriate content.[cite: 1]                       |
| <b>Intrusion Detection</b>    | Identify malware command-and-control traffic.[cite: 1]                  |
| <b>Network Analytics</b>      | Profile bandwidth usage per application type.[cite: 1]                  |
| <b>Content Filtering</b>      | Block domains listed as malicious by threat intelligence.[cite: 1]      |

## 2 Networking Fundamentals

This section establishes the networking theory required to understand how packets are structured, transmitted, and parsed.[cite: 1]

### 2.1 The OSI Model and Network Layering

All network communication is conceptually organized into **layers**, where each layer provides a service to the layer above and uses services from the layer below.[cite: 1] The most widely referenced model is the **OSI (Open Systems Interconnection)** model.[cite: 1]

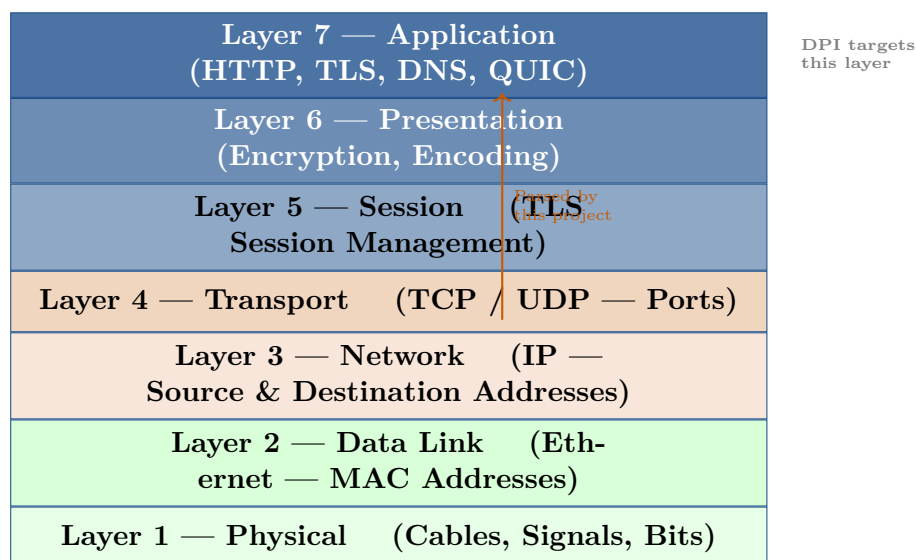


Figure 1: OSI Model — layers targeted by the DPI engine are highlighted[cite: 1]

In practice, the **TCP/IP model** (a simplified 4-layer version) is used:[cite: 1]

| TCP/IP Layer          | OSI Equivalent | Example Protocols       |
|-----------------------|----------------|-------------------------|
| <b>Application</b>    | Layers 5–7     | HTTP, TLS, DNS[cite: 1] |
| <b>Transport</b>      | Layer 4        | TCP, UDP[cite: 1]       |
| <b>Internet</b>       | Layer 3        | IPv4, IPv6[cite: 1]     |
| <b>Network Access</b> | Layers 1–2     | Ethernet[cite: 1]       |

The DPI engine parses packets from the **bottom up**: it first decodes the Ethernet frame, then the IP datagram, then the TCP/UDP segment, and finally inspects the application payload.[cite: 1]

## 2.2 The Ethernet Frame (Layer 2)

Every packet on a local network is wrapped in an **Ethernet frame**. [cite: 1] The Ethernet header is always **14 bytes** long.[cite: 1]

Ethernet Header — 14 bytes

|                     |                    |                      |                          |
|---------------------|--------------------|----------------------|--------------------------|
| Dest MAC<br>6 bytes | Src MAC<br>6 bytes | EtherType<br>2 bytes | Payload<br>(IP Datagram) |
|---------------------|--------------------|----------------------|--------------------------|

Figure 2: Ethernet frame structure[cite: 1]

**MAC Addresses.** A MAC (Media Access Control) address is a 6-byte hardware identifier assigned to each network interface card.[cite: 1] It is written in hexadecimal notation, e.g. AA:BB:CC:DD:EE:FF.[cite: 1]

**EtherType.** A 2-byte field indicating the type of payload carried by the frame:[cite: 1]

| EtherType Value | Protocol                                   |
|-----------------|--------------------------------------------|
| 0x0800          | IPv4[cite: 1]                              |
| 0x86DD          | IPv6[cite: 1]                              |
| 0x0806          | ARP (Address Resolution Protocol)[cite: 1] |

### 2.2.1 Ethernet Parsing in Source Code

The project defines the Ethernet header as a C++ struct in `include/packet_parser.h`: [cite: 1]

```

1 struct EthernetHeader {
2     std::array<uint8_t, 6> dest_mac;    // Destination MAC address
3     std::array<uint8_t, 6> src_mac;     // Source MAC address
4     uint16_t ether_type;               // 0x0800 = IPv4, 0x86DD = IPv6
5 };

```

Listing 1: Ethernet header struct (packet\_parser.h)[cite: 1]

The parser casts a raw byte pointer directly to this struct (using `reinterpret_cast`) after confirming sufficient bytes are available, exploiting the fact that the header fields have no padding at the beginning of the frame.[cite: 1]



## 2.3 The IPv4 Datagram (Layer 3)

When the EtherType is 0x0800, the Ethernet payload contains an **IPv4 datagram**.<sup>[cite: 1]</sup> The minimum IP header size is **20 bytes**.<sup>[cite: 1]</sup>

IPv4 Header (row 1–2)

|                             |               |                                  |               |                       |               |                 |                     |
|-----------------------------|---------------|----------------------------------|---------------|-----------------------|---------------|-----------------|---------------------|
| Ver+IHL<br>1 byte           | ToS<br>1 byte | TotalLen<br>2 bytes              | ID<br>2 bytes | Flags+Frag<br>2 bytes | TTL<br>1 byte | Proto<br>1 byte | Checksum<br>2 bytes |
| Source IP Address - 4 bytes |               | Destination IP Address - 4 bytes |               | Payload (TCP/UDP)     |               |                 |                     |

Figure 3: IPv4 header structure (simplified)<sup>[cite: 1]</sup>

### 2.3.1 Key Fields

| Field                | Size    | Description                                                                                                                          |
|----------------------|---------|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>Version + IHL</b> | 1 byte  | Upper 4 bits = IP version (4); lower 4 bits = header length in 32-bit words. A value of 5 means 20 bytes (5×4). <sup>[cite: 1]</sup> |
| <b>Protocol</b>      | 1 byte  | Identifies the encapsulated transport protocol: 1=ICMP, 6=TCP, 17=UDP. <sup>[cite: 1]</sup>                                          |
| <b>TTL</b>           | 1 byte  | Time To Live — decremented at each router hop. Packet is dropped when it reaches 0 (prevents routing loops). <sup>[cite: 1]</sup>    |
| <b>Source IP</b>     | 4 bytes | Sender's IP address in big-endian (network byte order). <sup>[cite: 1]</sup>                                                         |
| <b>Dest IP</b>       | 4 bytes | Recipient's IP address. <sup>[cite: 1]</sup>                                                                                         |

**Byte Order:** Network protocols use *big-endian* byte order (most significant byte first), while most PC processors use *little-endian*.<sup>[cite: 1]</sup> The source code must swap multi-byte fields using `ntohs()` (network-to-host short) and `ntohl()` (network-to-host long) when reading them.<sup>[cite: 1]</sup>

### 2.3.2 IPv4 Header in Source Code

```

1 struct IPv4Header {
2     uint8_t version_ihl;    // Version (4 bits) + Header Length (4
                             // bits)
3     uint8_t tos;           // Type of Service
4     uint16_t total_length;  // Total packet length (header + data)
5     uint16_t identification; // Fragment identification
6     uint16_t flags_fragment; // Flags (3 bits) + Fragment Offset (13
                             // bits)
7     uint8_t ttl;           // Time To Live
8     uint8_t protocol;      // TCP=6, UDP=17, ICMP=1
9     uint16_t checksum;     // Header checksum
10    uint32_t src_ip;        // Source IP address

```

```

11     uint32_t dest_ip;           // Destination IP address
12     // Options may follow if header length > 5 words
13 };

```

Listing 2: IPv4 header struct (packet\_parser.h)[cite: 1]

To extract the header length in bytes, the parser computes:[cite: 1]

$$(\text{version\_ihl} \& 0x0F) * 4$$

This masks the lower 4 bits (IHL) and multiplies by 4, since IHL is measured in 32-bit (4-byte) words.[cite: 1]

## 2.4 TCP — Transmission Control Protocol (Layer 4)

When the IPv4 protocol field is 6, the IP payload is a **TCP segment**. [cite: 1] TCP provides a reliable, ordered, connection-oriented byte stream.[cite: 1]

### 2.4.1 TCP Header Structure

The TCP header is at least **20 bytes** long:[cite: 1]

|                         |                     |                       |                       |                    |
|-------------------------|---------------------|-----------------------|-----------------------|--------------------|
| Src Port<br>2 bytes     | Dst Port<br>2 bytes | Seq Number<br>4 bytes | Ack Number<br>4 bytes |                    |
|                         |                     |                       |                       |                    |
| Offset+Flags<br>2 bytes | Window<br>2 bytes   | Checksum<br>2 bytes   | Urgent Ptr<br>2 bytes | Payload (App Data) |

Figure 4: TCP segment header[cite: 1]

### 2.4.2 TCP Flags

The flags byte is a critical field — it controls the state of the connection:[cite: 1]

| Flag       | Bit Mask | Meaning                                                    |
|------------|----------|------------------------------------------------------------|
| <b>FIN</b> | 0x01     | No more data from sender (connection closing)[cite: 1]     |
| <b>SYN</b> | 0x02     | Synchronize sequence numbers (connection opening)[cite: 1] |
| <b>RST</b> | 0x04     | Reset the connection (abort)[cite: 1]                      |
| <b>PSH</b> | 0x08     | Push data to application immediately[cite: 1]              |
| <b>ACK</b> | 0x10     | Acknowledgment field is valid[cite: 1]                     |
| <b>URG</b> | 0x20     | Urgent pointer field is significant[cite: 1]               |

These constants are defined in the project as:[cite: 1]

```

1 namespace TCPFlags {
2     constexpr uint8_t FIN = 0x01;
3     constexpr uint8_t SYN = 0x02;
4     constexpr uint8_t RST = 0x04;
5     constexpr uint8_t PSH = 0x08;
6     constexpr uint8_t ACK = 0x10;
7     constexpr uint8_t URG = 0x20;
8 }

```

Listing 3: TCP flag constants (packet\_parser.h)[cite: 1]

### 2.4.3 The TCP Three-Way Handshake

Before any application data flows, TCP establishes a connection via the **three-way handshake**:[\[cite: 1\]](#)

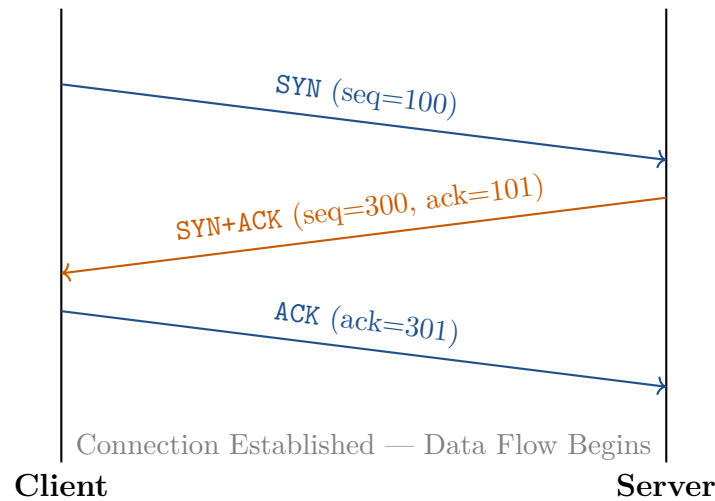


Figure 5: TCP three-way handshake[\[cite: 1\]](#)

The DPI engine tracks this handshake via the `syn_seen` and `syn_ack_seen` flags in the `Connection` struct.[\[cite: 1\]](#)

## 2.5 UDP — User Datagram Protocol (Layer 4)

When the IPv4 protocol field is 17, the IP payload is a **UDP datagram**.[\[cite: 1\]](#) Unlike TCP, UDP is *connectionless* and provides no delivery guarantees.[\[cite: 1\]](#) Its header is always exactly **8 bytes**:[\[cite: 1\]](#)

```

1 struct UDPHeader {
2     uint16_t src_port;    // Source port
3     uint16_t dest_port;   // Destination port
4     uint16_t length;      // Length of UDP header + data
5     uint16_t checksum;    // Checksum (optional in IPv4)
6 };
  
```

Listing 4: UDP header struct (`packet_parser.h`)[\[cite: 1\]](#)

DNS queries (port 53), QUIC (port 443 UDP), and DHCP all use UDP.[\[cite: 1\]](#)

## 2.6 The Five-Tuple: Identifying a Network Flow

A **flow** (also called a connection or session) is uniquely identified by five values called the **five-tuple**:[\[cite: 1\]](#)

| Field            | Example        | Purpose                                         |
|------------------|----------------|-------------------------------------------------|
| Source IP        | 192.168.1.100  | Who is sending <a href="#">[cite: 1]</a>        |
| Destination IP   | 172.217.14.206 | Where it is going <a href="#">[cite: 1]</a>     |
| Source Port      | 54321          | Sender's socket <a href="#">[cite: 1]</a>       |
| Destination Port | 443            | Service (443 = HTTPS) <a href="#">[cite: 1]</a> |
| Protocol         | TCP (6)        | TCP or UDP <a href="#">[cite: 1]</a>            |

All packets sharing the same five-tuple belong to the same application session.[cite: 1] The DPI engine uses consistent hashing of the five-tuple to route packets from the same flow to the same processing thread — ensuring stateful inspection is possible without thread contention.[cite: 1]

The five-tuple is defined in `include/types.h`:[cite: 1]

```

1 struct FiveTuple {
2     uint32_t src_ip;
3     uint32_t dst_ip;
4     uint16_t src_port;
5     uint16_t dst_port;
6     uint8_t protocol; // TCP=6, UDP=17
7
8     bool operator==(const FiveTuple& other) const {
9         return src_ip == other.src_ip && dst_ip == other.dst_ip &&
10             src_port == other.src_port && dst_port == other.
11                 dst_port &&
12                 protocol == other.protocol;
13     }
14
15     // Create reverse tuple (for bidirectional flow matching)
16     FiveTuple reverse() const {
17         return {dst_ip, src_ip, dst_port, src_port, protocol};
18     };

```

Listing 5: FiveTuple struct (types.h)[cite: 1]

A custom hash function (`FiveTupleHash`) is also provided, using the **Fibonacci hashing** constant `0x9e3779b9` to combine field hashes with low collision probability.[cite: 1]

## 2.7 The PCAP File Format

**PCAP** (Packet Capture) is the standard binary file format for storing captured network traffic.[cite: 1] It is produced by tools like Wireshark, tcpdump, and similar utilities.[cite: 1] The format consists of:[cite: 1]

1. A **24-byte Global Header** written once at the start of the file.[cite: 1]
2. A sequence of (**16-byte Packet Header** + **Packet Data**) records.[cite: 1]

### 2.7.1 Global Header (24 bytes)

| Field                     | Size    | Description                                                       |
|---------------------------|---------|-------------------------------------------------------------------|
| <b>Magic Number</b>       | 4 bytes | 0xd4c3b2a1 (little-endian) or 0xa1b2c3d4 (big-endian). Identifier |
| <b>Version Major</b>      | 2 bytes | Major version number (currently 2).[cite: 1]                      |
| <b>Version Minor</b>      | 2 bytes | Minor version number (currently 4).[cite: 1]                      |
| <b>Timezone Offset</b>    | 4 bytes | Usually 0 (UTC).[cite: 1]                                         |
| <b>Timestamp Accuracy</b> | 4 bytes | Usually 0.[cite: 1]                                               |
| <b>Snap Length</b>        | 4 bytes | Maximum captured packet length (e.g. 65535).[cite: 1]             |
| <b>Link Type</b>          | 4 bytes | Data link type. 1 = Ethernet.[cite: 1]                            |

### 2.7.2 Per-Packet Header (16 bytes)

| Field                         | Size    | Description                                                   |
|-------------------------------|---------|---------------------------------------------------------------|
| <b>Timestamp Seconds</b>      | 4 bytes | Capture time — seconds since Unix epoch.[cite: 1]             |
| <b>Timestamp Microseconds</b> | 4 bytes | Sub-second precision.[cite: 1]                                |
| <b>Captured Length</b>        | 4 bytes | Number of bytes stored in the file for this packet.[cite: 1]  |
| <b>Original Length</b>        | 4 bytes | Actual packet length (may differ if packet was truncated).[ci |

Immediately following the packet header are `captured_length` bytes of raw packet data, starting from the Ethernet frame.[cite: 1]

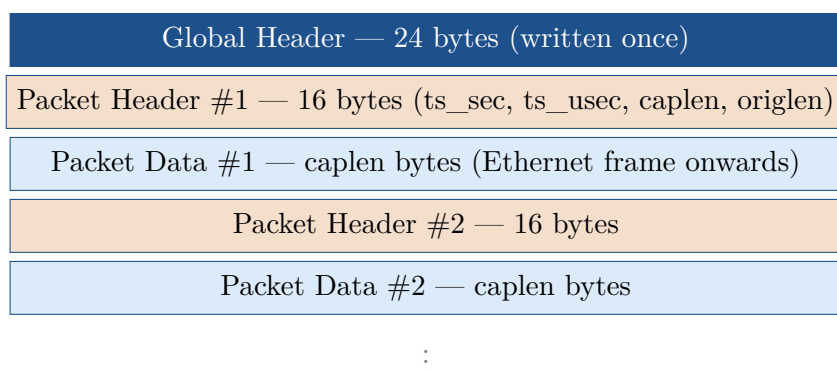


Figure 6: PCAP file layout[cite: 1]

### 2.7.3 PCAP Structures in Source Code

The project defines these in `include/pcap_reader.h`: [cite: 1]

```

1 struct PcapGlobalHeader {
2     uint32_t magic_number;    // 0xd4c3b2a1 (little-endian files)
3     uint16_t version_major;   // 2
4     uint16_t version_minor;  // 4
5     int32_t  thiszone;        // Timezone offset (usually 0)
6     uint32_t sigfigs;         // Timestamp accuracy (usually 0)
7     uint32_t snaplen;         // Max captured packet size
8     uint32_t network;         // Link-layer type (1 = Ethernet)
9 };
10
11 struct PcapPacketHeader {
12     uint32_t ts_sec;          // Timestamp seconds
13     uint32_t ts_usec;         // Timestamp microseconds
14     uint32_t incl_len;        // Bytes captured in file
15     uint32_t orig_len;        // Actual packet size
16 };
17
18 struct RawPacket {
19     uint32_t ts_sec;
20     uint32_t ts_usec;
21     uint32_t captured_length;
22     uint32_t original_length;
23     std::vector<uint8_t> data; // The raw bytes
24 };
  
```

Listing 6: ]PCAP data structures (`pcap_reader.h`) [cite: 1]

## 3 Deep Packet Inspection — Theory

### 3.1 What is Deep Packet Inspection?

Traditional packet filtering (as used in simple firewalls) operates at **Layers 3–4** only.[cite: 1] It makes allow/deny decisions based solely on:[cite: 1]

- Source and destination IP addresses[cite: 1]
- Source and destination port numbers[cite: 1]
- Protocol (TCP or UDP)[cite: 1]

**Deep Packet Inspection (DPI)** goes further: it examines the **application-layer payload** — the actual content inside the TCP/UDP segment.[cite: 1] This allows the engine to:[cite: 1]

- Identify *which application* is generating the traffic (YouTube vs. Zoom vs. GitHub), even when both use port 443 (HTTPS).[cite: 1]
- Detect protocols that are deliberately disguised or use non-standard ports.[cite: 1]
- Extract metadata such as the destination domain name.[cite: 1]
- Apply policy decisions per-application rather than per-port.[cite: 1]

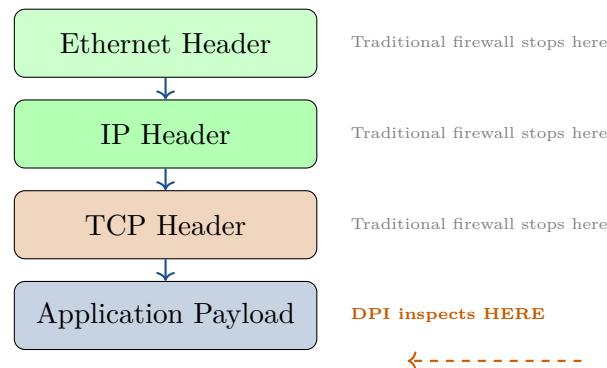


Figure 7: Traditional firewall vs. DPI inspection depth[cite: 1]

### 3.2 The HTTPS Paradox

A common misconception is that HTTPS (HTTP over TLS) prevents all traffic analysis.[cite: 1] While TLS *does* encrypt the HTTP request and response bodies, a portion of the TLS handshake occurs **before** encryption is negotiated.[cite: 1]

The most important piece of unencrypted information is the **Server Name Indication (SNI)**, transmitted in the TLS Client Hello.[cite: 1]

**Key insight:** When a browser connects to `https://www.youtube.com`, the very first packet sent contains the string `www.youtube.com` in **plaintext**. [cite: 1] Anyone intercepting the network can read the destination domain without decrypting

anything.[cite: 1]

### 3.3 The TLS Handshake

TLS (Transport Layer Security) establishes an encrypted session through a **handshake** — a series of messages exchanged before any application data is sent.[cite: 1]

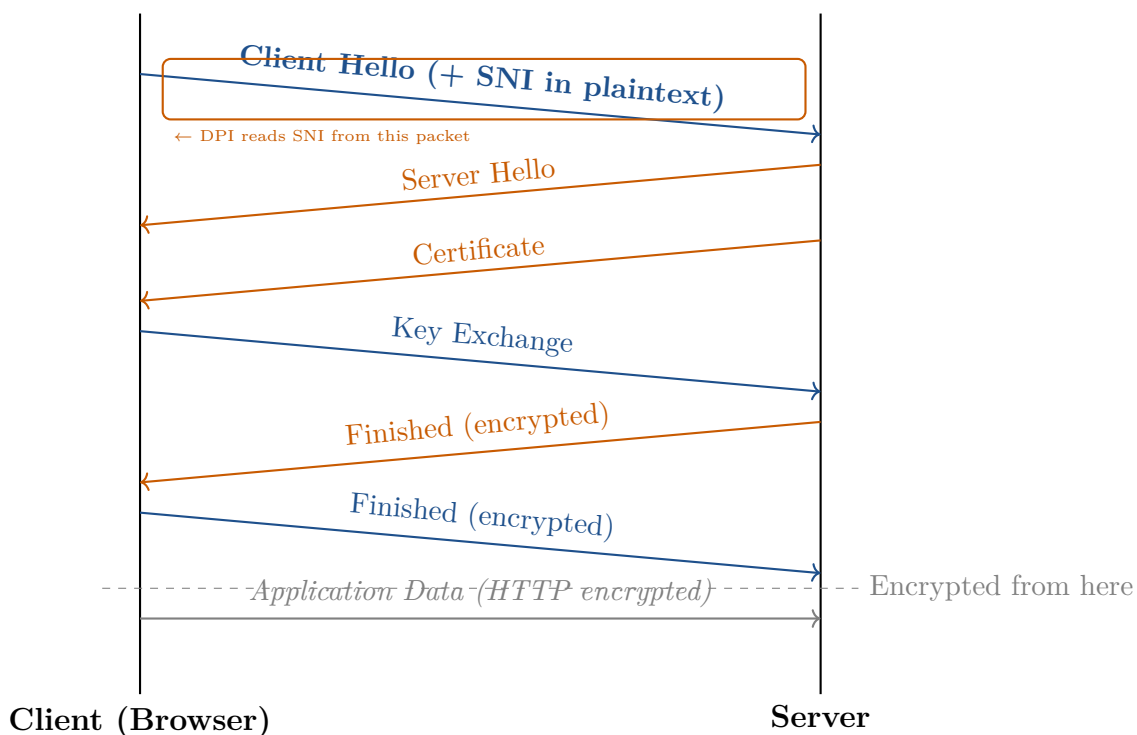


Figure 8: TLS handshake — SNI is visible in the Client Hello[cite: 1]

### 3.4 TLS Client Hello Structure

The TLS Client Hello is the first message sent by the client.[cite: 1] Its internal binary structure is as follows:[cite: 1]

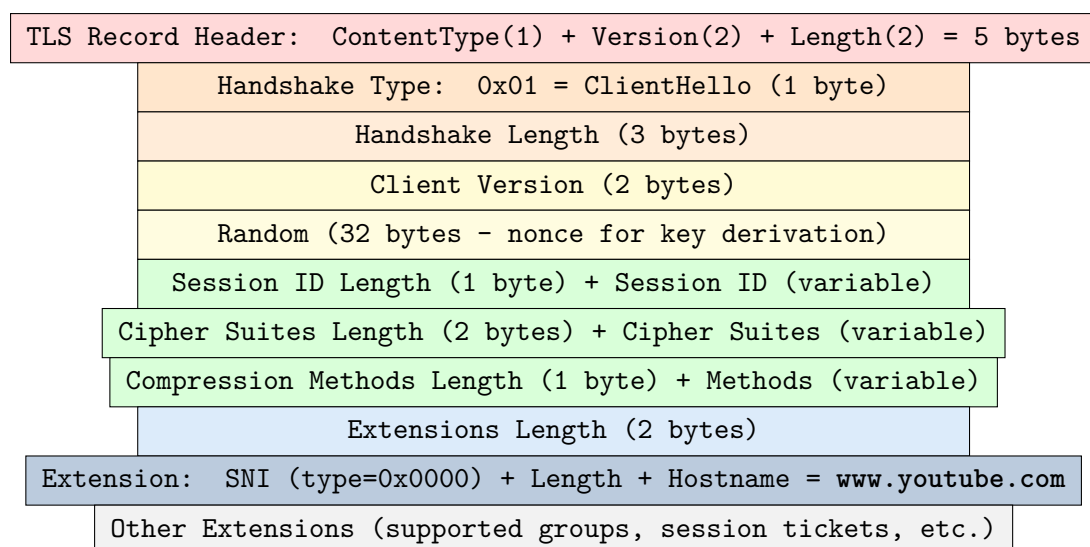


Figure 9: TLS Client Hello binary layout — SNI extension highlighted[cite: 1]

### 3.4.1 SNI Extension Binary Format

Within the extensions block, the SNI extension has the following format:[cite: 1]

| Field                   | Size     | Value                                  |
|-------------------------|----------|----------------------------------------|
| <b>Extension Type</b>   | 2 bytes  | 0x0000 (SNI)[cite: 1]                  |
| <b>Extension Length</b> | 2 bytes  | Total length of SNI data[cite: 1]      |
| <b>SNI List Length</b>  | 2 bytes  | Length of the SNI entry list[cite: 1]  |
| <b>SNI Type</b>         | 1 byte   | 0x00 (host_name)[cite: 1]              |
| <b>SNI Length</b>       | 2 bytes  | Length of the hostname string[cite: 1] |
| <b>SNI Value</b>        | variable | e.g. www.youtube.com[cite: 1]          |

## 3.5 HTTP Host Header (Unencrypted Traffic)

For plain HTTP (port 80, unencrypted), the destination domain is visible in the **Host** header of the HTTP request:[cite: 1]

Listing 7: ]Example HTTP request showing Host header[cite: 1]

```
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html
```

The project's `HTTPHostExtractor` class searches for the string "Host: " in the TCP payload and extracts everything up to the line terminator (`\r\n`).[cite: 1]

## 3.6 DNS Query Extraction

DNS (Domain Name System) runs over UDP port 53.[cite: 1] A DNS query reveals which domains a host is looking up.[cite: 1] The `DNSExtractor` class parses the DNS wire format to extract the queried name, enabling the engine to correlate IP connections with domain names even before the TCP connection is established.[cite: 1]



### 3.7 QUIC Protocol

QUIC is a modern transport protocol developed by Google, used for HTTP/3.[cite: 1] It runs over **UDP port 443** and embeds TLS 1.3 in its own framing.[cite: 1] QUIC Initial packets still contain a TLS Client Hello (within CRYPTO frames), making SNI extraction possible.[cite: 1] The `QUICSNIExtractor` class handles this more complex case.[cite: 1]

### 3.8 Application Classification

Once an SNI or Host header is extracted, the engine maps it to a known application type using `sniToAppType()` in `types.cpp`. [cite: 1] The supported application types are defined as an enum:[cite: 1]

```
1 enum class AppType {
2     UNKNOWN = 0, HTTP, HTTPS, DNS, TLS, QUIC,
3     GOOGLE, FACEBOOK, YOUTUBE, TWITTER, INSTAGRAM,
4     NETFLIX, AMAZON, MICROSOFT, APPLE, WHATSAPP,
5     TELEGRAM, TIKTOK, SPOTIFY, ZOOM, DISCORD,
6     GITHUB, CLOUDFLARE,
7     APP_COUNT // Keep this last for counting
8 };
```

Listing 8: Application type enum (types.h)[cite: 1]

The mapping from SNI domain to `AppType` is done by substring matching.[cite: 1] For example, any SNI containing `youtube` or `googlevideo` maps to `AppType::YOUTUBE`. [cite: 1]

| SNI Contains         | App Type           |
|----------------------|--------------------|
| youtube, googlevideo | YOUTUBE[cite: 1]   |
| netflix, nflxvideo   | NETFLIX[cite: 1]   |
| facebook, fbcdn      | FACEBOOK[cite: 1]  |
| instagram            | INSTAGRAM[cite: 1] |
| tiktok, musical.ly   | TIKTOK[cite: 1]    |
| zoom.us              | ZOOM[cite: 1]      |
| discord              | DISCORD[cite: 1]   |
| github               | GITHUB[cite: 1]    |

### 3.9 Connection States

Each tracked connection transitions through a defined set of states:[cite: 1]

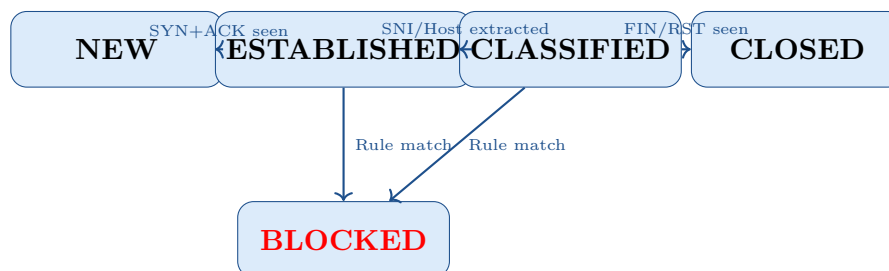


Figure 10: Connection state machine[cite: 1]

### 3.10 Packet Actions

After inspection, the engine assigns one of four possible actions to each packet:[cite: 1]

| Action          | Description                                               |
|-----------------|-----------------------------------------------------------|
| <b>FORWARD</b>  | Send to the output PCAP (permit)[cite: 1]                 |
| <b>DROP</b>     | Silently discard (block)[cite: 1]                         |
| <b>INSPECT</b>  | Needs further inspection (more payload expected)[cite: 1] |
| <b>LOG_ONLY</b> | Forward but record for audit purposes[cite: 1]            |

## 4 Project Architecture

### 4.1 Two-Mode Design

The project implements two distinct processing pipelines sharing the same parsing and classification logic:[cite: 1]

#### 4.1.1 Simple Single-Threaded Mode

`src/main_working.cpp` implements a straightforward loop:[cite: 1]

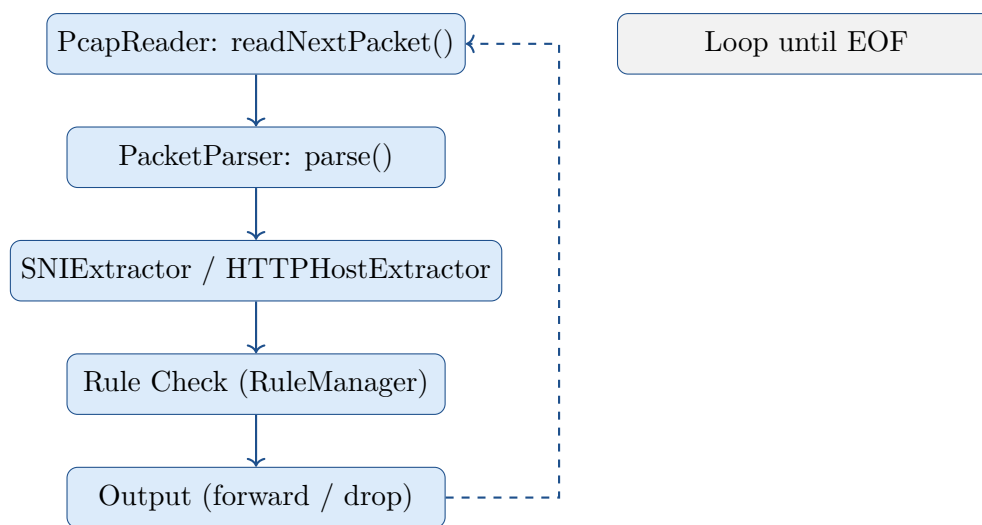


Figure 11: Single-threaded processing pipeline[cite: 1]

#### 4.1.2 Multi-Threaded DPI Mode

`src/dpi_mt.cpp` and the `DPIEngine` class implement a parallel pipeline with multiple threads:[cite: 1]

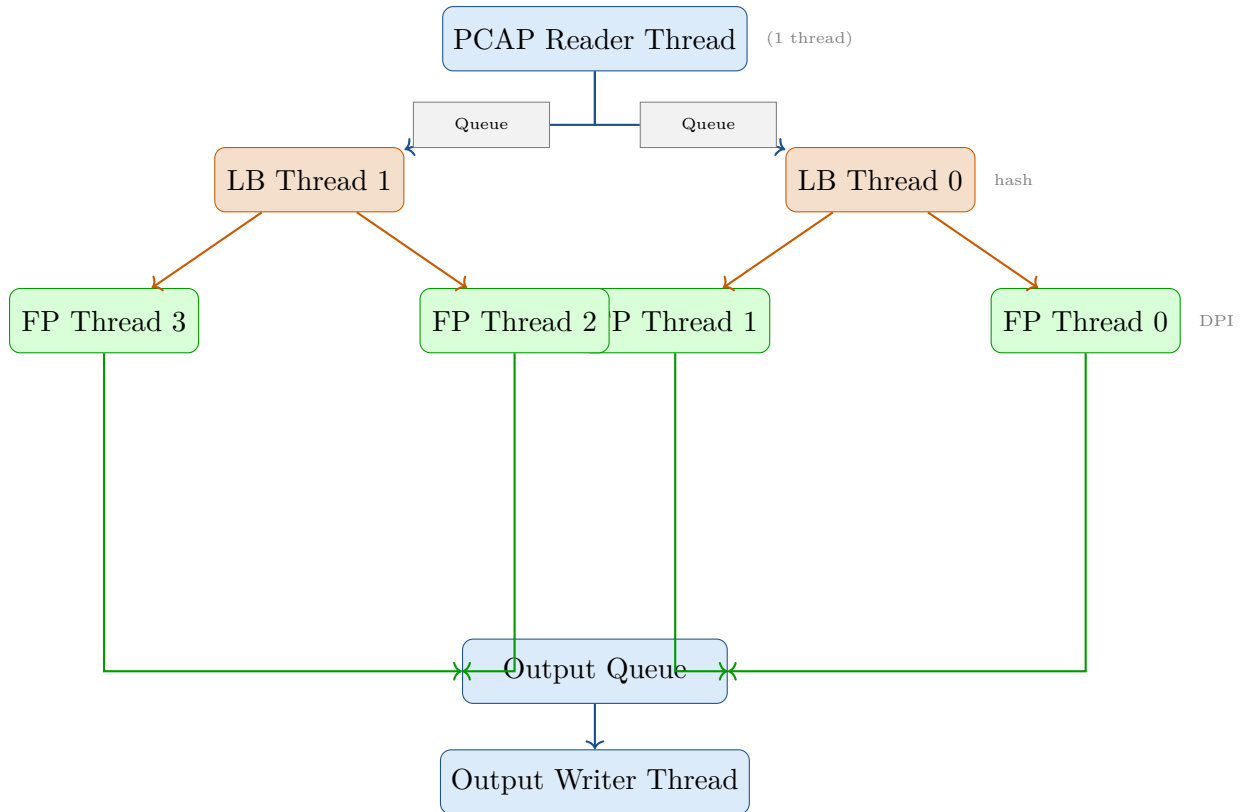


Figure 12: Multi-threaded DPI Engine thread architecture[cite: 1]

## 4.2 Thread Roles

| Component          | Threads | Responsibility                                                                                                        |
|--------------------|---------|-----------------------------------------------------------------------------------------------------------------------|
| PCAP Reader        | 1       | Reads raw packets from the input file; distributes to LB input queues.[cite: 1]                                       |
| Load Balancer (LB) | 2       | Receives packets from reader; hashes five-tuple to select target FP; enqueues to FP input queue.[cite: 1]             |
| Fast Path (FP)     | 4       | Performs connection tracking, payload inspection, SNI/Host extraction, rule matching, and forwards or drops.[cite: 1] |
| Output Writer      | 1       | Dequeues forwarded packets from the output queue and writes them to the output PCAP file.[cite: 1]                    |

## 4.3 Why Consistent Hashing?

The most critical design requirement is that **all packets belonging to the same five-tuple flow must go to the same FP thread**. [cite: 1] This is because: [cite: 1]

1. Connection tracking state (TCP flags seen, SNI extracted, bytes counted) is stored per-FP without locking — there is no shared state. [cite: 1]

2. If two packets from the same flow went to different FP threads, each thread would see only half the flow and could not reconstruct the TLS handshake or track byte counts correctly.[cite: 1]

The two-level hashing scheme achieves this:[cite: 1]

$$\text{LB index} = \text{hash}(\text{five-tuple}) \bmod N_{LB}$$

$$\text{FP index} = \text{hash}(\text{five-tuple}) \bmod N_{FP/LB} \quad (\text{within that LB's pool})$$

Since the hash is deterministic, the same five-tuple always maps to the same FP thread.[cite: 1]

## 4.4 Thread-Safe Queue

Inter-thread communication uses a **bounded, blocking, thread-safe queue** defined in `include/thread_safe_queue.h`. [cite: 1] Key design points:[cite: 1]

- **Bounded** (default: 10,000 items) — backpressure prevents memory exhaustion if FP threads fall behind.[cite: 1]
- **Blocking push** — the reader/LB block when the queue is full, creating natural flow control.[cite: 1]
- **Timed pop** — FP threads use `popWithTimeout()` so they can periodically clean up stale connections even when no packets arrive.[cite: 1]
- **Shutdown signal** — `shutdown()` wakes all blocked threads so the engine can terminate cleanly.[cite: 1]

```

1 std::optional<T> popWithTimeout(std::chrono::milliseconds timeout) {
2     std::unique_lock<std::mutex> lock(mutex_);
3     if (!not_empty_.wait_for(lock, timeout,
4         [this] { return !queue_.empty() || shutdown_; }))) {
5         return std::nullopt; // Timeout      no item available
6     }
7     if (queue_.empty()) return std::nullopt; // Shutdown signal
8     T item = std::move(queue_.front());
9     queue_.pop();
10    not_full_.notify_one(); // Wake blocked producers
11    return item;
12 }
```

Listing 9: Thread-safe queue pop with timeout (`thread_safe_queue.h`) [cite: 1]

The use of `std::condition_variable` with `wait_for` and a predicate lambda correctly handles spurious wakeups — a common pitfall in multi-threaded programming.[cite: 1]

## 5 Component Deep-Dives

This section examines each source-code component in detail, explaining both *what* it does and *how* the code achieves it.[cite: 1]

## 5.1 PcapReader — src/pcap\_reader.cpp

### 5.1.1 Responsibility

The PcapReader class encapsulates all file I/O for reading PCAP files.[cite: 1] It provides a simple iterator-style interface: callers call readNextPacket() in a loop until it returns false.[cite: 1]

### 5.1.2 Open and Validate

```

1 bool PcapReader::open(const std::string& filename) {
2     file_.open(filename, std::ios::binary);
3     if (!file_.is_open()) return false;
4
5     // Read and validate the 24-byte global header
6     file_.read(reinterpret_cast<char*>(&global_header_), sizeof(
7         global_header_));
8
9     // Magic number check: 0xd4c3b2a1 = little-endian PCAP
10    if (global_header_.magic_number != 0xd4c3b2a1) {
11        // Could also be 0xa1b2c3d4 (big-endian) not handled here
12        std::cerr << "Invalid PCAP magic number\n";
13        return false;
14    }
15    return true;
16 }
```

Listing 10: PcapReader::open — opening and validating a PCAP file[cite: 1]

#### Key implementation details:

- The file is opened in std::ios::binary mode — crucial on Windows where text mode would transform \r\n sequences, corrupting binary data.[cite: 1]
- reinterpret\_cast<char\*> is used to treat the struct as a byte array for std::ifstream::read().[cite: 1]
- Only the little-endian magic number is checked, which covers virtually all modern PCAP files.[cite: 1]

### 5.1.3 Reading Packets

```

1 bool PcapReader::readNextPacket(RawPacket& packet) {
2     PcapPacketHeader pkt_header;
3     file_.read(reinterpret_cast<char*>(&pkt_header), sizeof(pkt_header));
4     if (file_.gcount() < sizeof(pkt_header)) return false; // EOF
5
6     packet.ts_sec      = pkt_header.ts_sec;
7     packet.ts_usec     = pkt_header.ts_usec;
8     packet.captured_length = pkt_header.incl_len;
9     packet.original_length = pkt_header.orig_len;
10
11    packet.data.resize(pkt_header.incl_len);
12 }
```

```

12     file_.read(reinterpret_cast<char*>(packet.data.data()), pkt_header
        .incl_len);
13     return file_.gcount() == pkt_header.incl_len;
14 }

```

Listing 11: ]PcapReader::readNextPacket[cite: 1]

Each call reads exactly one packet: first the 16-byte header, then `incl_len` bytes of raw data into a `std::vector<uint8_t>`.[\[cite: 1\]](#) The vector takes ownership of the memory, which is later passed by value into the processing pipeline.[\[cite: 1\]](#)

## 5.2 PacketParser — src/packet\_parser.cpp

### 5.2.1 Responsibility

PacketParser is a stateless utility class (all methods are `static`) that decodes raw byte arrays into structured `ParsedPacket` objects, layer by layer.[\[cite: 1\]](#)

### 5.2.2 Top-Level Parse Method

```

1  bool PacketParser::parse(const RawPacket& raw, ParsedPacket& parsed) {
2      if (raw.data.empty()) return false;
3
4      parsed.timestamp_sec  = raw.ts_sec;
5      parsed.timestamp_usec = raw.ts_usec;
6
7      const uint8_t* data = raw.data.data();
8      size_t len        = raw.data.size();
9      size_t offset = 0;
10
11     // Layer 2: Ethernet
12     if (!parseEthernet(data, len, parsed, offset)) return false;
13
14     // Layer 3: IPv4 (if EtherType matches)
15     if (parsed.ether_type == EtherType::IPv4) {
16         if (!parseIPv4(data, len, parsed, offset)) return false;
17     }
18
19     // Layer 4: TCP or UDP
20     if (parsed.has_ip) {
21         if (parsed.protocol == Protocol::TCP)
22             parseTCP(data, len, parsed, offset);
23         else if (parsed.protocol == Protocol::UDP)
24             parseUDP(data, len, parsed, offset);
25     }
26
27     return true;
28 }

```

Listing 12: ]PacketParser::parse — orchestrates layer parsing[cite: 1]

The `offset` variable accumulates as each layer is parsed, pointing to the start of the next layer's data.[\[cite: 1\]](#)

### 5.2.3 Ethernet Parsing

```

1 bool PacketParser::parseEthernet(const uint8_t* data, size_t len,
2                                ParsedPacket& parsed, size_t& offset
3                                ) {
4     if (len < 14) return false; // Minimum Ethernet header size
5
6     const EthernetHeader* eth =
7         reinterpret_cast<const EthernetHeader*>(data + offset);
8
9     parsed.src_mac = macToString(eth->src_mac.data());
10    parsed.dest_mac = macToString(eth->dest_mac.data());
11    parsed.ether_type = ntohs(eth->ether_type); // Convert to host
12                                                byte order
13
14    offset += 14; // Advance past the Ethernet header
15    return true;
16 }

```

Listing 13: ]PacketParser::parseEthernet[cite: 1]

**Byte order conversion:** `ntohs()` (network-to-host short) converts the 16-bit Ether-Type from big-endian network order to the host’s native byte order for correct comparison.[cite: 1]

#### 5.2.4 IPv4 Parsing

```

1 bool PacketParser::parseIPv4(const uint8_t* data, size_t len,
2                              ParsedPacket& parsed, size_t& offset) {
3     if (offset + 20 > len) return false;
4
5     const IPv4Header* ip =
6         reinterpret_cast<const IPv4Header*>(data + offset);
7
8     parsed.has_ip = true;
9     parsed.ip_version = (ip->version_ihl >> 4); // Upper 4 bits =
10                                                version
11     parsed.src_ip = ipToString(ntohl(ip->src_ip));
12     parsed.dest_ip = ipToString(ntohl(ip->dest_ip));
13     parsed.protocol = ip->protocol;
14     parsed.ttl = ip->ttl;
15
16     // Header length in bytes: lower 4 bits of version_ihl * 4
17     uint8_t header_len = (ip->version_ihl & 0x0F) * 4;
18     offset += header_len; // Skip header (may include IP options)
19     return true;
20 }

```

Listing 14: ]PacketParser::parseIPv4 (key parts)[cite: 1]

#### 5.2.5 TCP Parsing

```

1 bool PacketParser::parseTCP(const uint8_t* data, size_t len,
2                             ParsedPacket& parsed, size_t& offset) {
3     if (offset + 20 > len) return false;
4

```

```

5     const TCPHeader* tcp =
6         reinterpret_cast<const TCPHeader*>(data + offset);
7
8     parsed.has_tcp      = true;
9     parsed.src_port     = ntohs(tcp->src_port);
10    parsed.dest_port     = ntohs(tcp->dest_port);
11    parsed.seq_number    = ntohl(tcp->seq_number);
12    parsed.ack_number    = ntohl(tcp->ack_number);
13    parsed.tcp_flags     = tcp->flags;
14
15    // TCP data offset (upper 4 bits of data_offset) in 32-bit words
16    uint8_t tcp_header_len = (tcp->data_offset >> 4) * 4;
17    offset += tcp_header_len;
18
19    // Remaining bytes are the payload
20    parsed.payload_data   = data + offset;
21    parsed.payload_length = (offset < len) ? (len - offset) : 0;
22
23    return true;
24 }

```

Listing 15: ]PacketParser::parseTCP (key parts)[cite: 1]

The payload pointer (`payload_data`) points *directly into* the original `RawPacket::data` vector — zero-copy design avoids unnecessary memory allocation for large payloads.[cite: 1]

### 5.2.6 Helper: IP Address to String

```

1 std::string PacketParser::ipToString(uint32_t ip) {
2     // ip is already in host byte order after ntohl()
3     return std::to_string((ip >> 24) & 0xFF) + "." +
4           std::to_string((ip >> 16) & 0xFF) + "." +
5           std::to_string((ip >> 8) & 0xFF) + "." +
6           std::to_string(ip & 0xFF);
7 }

```

Listing 16: ]PacketParser::ipToString[cite: 1]

## 5.3 SNI Extractor — `src/sni_extractor.cpp`

### 5.3.1 Responsibility

The `SNIExtractor` class is the heart of the DPI functionality.[cite: 1] It navigates the binary TLS Client Hello structure to find and extract the Server Name Indication host-name.[cite: 1]

### 5.3.2 Detection: Is This a TLS Client Hello?

```

1 bool SNIExtractor::isTLSClientHello(const uint8_t* payload, size_t
   length) {
2     if (length < 9) return false;
3
4     // Byte 0: Content Type must be 0x16 (Handshake)
5     if (payload[0] != CONTENT_TYPE_HANDSHAKE) return false;

```



```

6
7    // Bytes 1-2: TLS version (0x0300 through 0x0304)
8    uint16_t version = readUint16BE(payload + 1);
9    if (version < 0x0300 || version > 0x0304) return false;
10
11    // Byte 5: Handshake type must be 0x01 (Client Hello)
12    if (payload[5] != HANDSHAKE_CLIENT_HELLO) return false;
13
14    return true;
15 }

```

Listing 17: ]SNIExtractor::isTLSClientHello[cite: 1]

This three-check sequence correctly identifies TLS Client Hello packets while rejecting other TLS record types (Server Hello, Application Data, etc.).[cite: 1]

### 5.3.3 Extraction: Navigating to the SNI

The full extraction walks the binary structure by maintaining a byte offset:[cite: 1]

```

1 std::optional<std::string> SNIExtractor::extract(
2     const uint8_t* payload, size_t length) {
3     if (!isTLSClientHello(payload, length)) return std::nullopt;
4
5     size_t offset = 5;           // Skip TLS record header (5 bytes)
6     offset += 4;                 // Skip handshake type + 3-byte
7     length                     // length
8     offset += 2;                 // Skip client version
9     offset += 32;                // Skip random (32 bytes)
10
11    // Skip Session ID
12    uint8_t sid_len = payload[offset++];
13    offset += sid_len;
14
15    // Skip Cipher Suites
16    uint16_t cs_len = readUint16BE(payload + offset);
17    offset += 2 + cs_len;
18
19    // Skip Compression Methods
20    uint8_t cm_len = payload[offset++];
21    offset += cm_len;
22
23    // Parse Extensions
24    uint16_t ext_len = readUint16BE(payload + offset);
25    offset += 2;
26    size_t ext_end = offset + ext_len;
27
28    while (offset + 4 <= ext_end) {
29        uint16_t ext_type = readUint16BE(payload + offset);
30        uint16_t ext_data_len = readUint16BE(payload + offset + 2);
31        offset += 4;
32
33        if (ext_type == EXTENSION_SNI) { // 0x0000
34            // SNI list length (2 bytes), SNI type (1 byte), SNI
35            // length (2 bytes)
36            offset += 3; // skip SNI list length + type
37            uint16_t sni_len = readUint16BE(payload + offset);
38            offset += 2;

```

```

37         // Extract the hostname string
38         return std::string(
39             reinterpret_cast<const char*>(payload + offset),
40             sni_len);
41     }
42     offset += ext_data_len; // Skip to next extension
43 }
44 return std::nullopt; // SNI extension not found
45 }
```

Listing 18: ]SNIExtractor::extract — key navigation steps[cite: 1]

### 5.3.4 HTTP Host Extraction

```

1 std::optional<std::string> HTTPHostExtractor::extract(
2     const uint8_t* payload, size_t length) {
3     // Cast payload to string_view for safe string operations
4     std::string_view data(reinterpret_cast<const char*>(payload),
5         length);
6
7     // Check for HTTP method
8     if (!isHTTPRequest(payload, length)) return std::nullopt;
9
10    // Search for "Host: " header
11    const std::string host_header = "Host: ";
12    auto pos = data.find(host_header);
13    if (pos == std::string_view::npos) return std::nullopt;
14
15    pos += host_header.size();
16    auto end = data.find("\r\n", pos);
17    if (end == std::string_view::npos) end = length;
18
19    // Strip port number if present (e.g., "example.com:8080")
20    std::string host(data.substr(pos, end - pos));
21    auto colon = host.find(':');
22    if (colon != std::string::npos) host = host.substr(0, colon);
23
24    return host;
25 }
```

Listing 19: ]HTTPHostExtractor::extract[cite: 1]

## 5.4 Rule Manager — src/rule\_manager.cpp

### 5.4.1 Responsibility

The `RuleManager` maintains the set of active blocking rules and provides thread-safe lookup methods.[cite: 1] It supports four rule categories: **IP**, **Application**, **Domain**, and **Port**. [cite: 1]

### 5.4.2 Thread Safety Design

Because FP threads concurrently read rules while the main thread (or an admin interface) may modify them, the `RuleManager` uses `std::shared_mutex` (C++17).[cite: 1]

```

1 bool RuleManager::isIPBlocked(uint32_t ip) const {
2     // shared_lock allows multiple concurrent readers
3     std::shared_lock<std::shared_mutex> lock(ip_mutex_);
4     return blocked_ips_.count(ip) > 0;
5 }
6
7 void RuleManager::blockIP(uint32_t ip) {
8     // unique_lock allows one exclusive writer
9     std::unique_lock<std::shared_mutex> lock(ip_mutex_);
10    blocked_ips_.insert(ip);
11 }

```

Listing 20: Thread-safe rule check (rule\_manager.cpp)[cite: 1]

This pattern — **read-write lock** — allows many FP threads to simultaneously read rules (most common operation) while writes are serialized and exclusive.[cite: 1]

### 5.4.3 Domain Wildcard Matching

Domain rules support wildcard patterns such as \*.facebook.com to block all subdomains:[cite: 1]

```

1 bool RuleManager::domainMatchesPattern(const std::string& domain,
2                                         const std::string& pattern) {
3     if (pattern.substr(0, 2) == "*.") {
4         // Wildcard: match any subdomain
5         std::string suffix = pattern.substr(1); // ".facebook.com"
6         return domain.size() >= suffix.size() &&
7             domain.compare(domain.size() - suffix.size(),
8                             suffix.size(), suffix) == 0;
9     }
10    return domain == pattern; // Exact match
11 }

```

Listing 21: Wildcard domain matching[cite: 1]

### 5.4.4 Combined Rule Check

The most important method combines all rule types in one pass:[cite: 1]

```

1 std::optional<RuleManager::BlockReason> RuleManager::shouldBlock(
2     uint32_t src_ip, uint16_t dst_port,
3     AppType app, const std::string& domain) const {
4     // 1. Check IP
5     if (isIPBlocked(src_ip))
6         return BlockReason{BlockReason::IP,
7                             "Source IP " + ipToString(src_ip) + " is
9                             blocked"};
10
11    // 2. Check port
12    if (isPortBlocked(dst_port))
13        return BlockReason{BlockReason::PORT,
14                            "Port " + std::to_string(dst_port) + " is
16                            blocked"};
17
18    // 3. Check app
19    if (app != AppType::UNKNOWN && isAppBlocked(app))
20        return BlockReason{BlockReason::APP,

```

```

15         "App " + appTypeToString(app) + " is
           blocked";
16     // 4. Check domain
17     if (!domain.empty() && isDomainBlocked(domain))
18         return BlockReason{BlockReason::DOMAIN,
19                             "Domain " + domain + " is blocked"};
20
21     return std::nullopt; // Not blocked
22 }

```

Listing 22: ]RuleManager::shouldBlock — combined check[cite: 1]

#### 5.4.5 Rule Persistence

Rules can be saved and loaded from a plain-text file for persistence across restarts.[cite: 1] The file format is one rule per line:[cite: 1]

Listing 23: ]Example rules file format[cite: 1]

```

# DPI Engine Block Rules
block_ip 192.168.1.50
block_app YOUTUBE
block_app TIKTOK
block_domain *.facebook.com
block_domain *.instagram.com
block_port 6881

```

## 5.5 Connection Tracker — src/connection\_tracker.cpp

### 5.5.1 Responsibility

The `ConnectionTracker` maintains a hash map of active network flows.[cite: 1] Each `FastPathProcessor` has its own private `ConnectionTracker` instance — no sharing is needed because consistent hashing guarantees that all packets from a given five-tuple flow go to the same FP thread.[cite: 1]

### 5.5.2 The Connection Hash Map

```

1 // Key: FiveTuple (with custom hash)
2 // Value: Connection struct (state, SNI, stats, ...)
3 std::unordered_map<FiveTuple, Connection, FiveTupleHash> connections_;

```

Listing 24: ]ConnectionTracker internal storage (connection\_tracker.h)[cite: 1]

### 5.5.3 Get or Create

```

1 Connection* ConnectionTracker::getOrCreateConnection(const FiveTuple&
  tuple) {
2     auto it = connections_.find(tuple);
3     if (it != connections_.end()) return &it->second;
4
5     // Evict oldest entry if table is at capacity
6     if (connections_.size() >= max_connections_) evictOldest();
7

```

```

8      // Create new connection
9      Connection conn;
10     conn.tuple      = tuple;
11     conn.state      = ConnectionState::NEW;
12     conn.first_seen = std::chrono::steady_clock::now();
13     conn.last_seen  = conn.first_seen;
14
15     auto result = connections_.emplace(tuple, std::move(conn));
16     total_seen_++;
17     return &result.first->second;
18 }

```

Listing 25: ]ConnectionTracker::getOrCreateConnection[cite: 1]

### 5.5.4 Bidirectional Flow Lookup

When looking up an existing connection, the tracker also checks the **reverse tuple** to handle response packets:[cite: 1]

```

1 Connection* ConnectionTracker::getConnection(const FiveTuple& tuple) {
2     auto it = connections_.find(tuple);
3     if (it != connections_.end()) return &it->second;
4
5     // Try reverse tuple (server -> client response packets)
6     auto rev = connections_.find(tuple.reverse());
7     if (rev != connections_.end()) return &rev->second;
8
9     return nullptr;
10 }

```

Listing 26: ]Bidirectional lookup (connection\_tracker.cpp)[cite: 1]

### 5.5.5 Stale Connection Cleanup

TCP connections can be abandoned without a proper FIN/RST (e.g. network failure).[cite: 1] The cleanup method is called periodically from the FP thread's idle path:[cite: 1]

```

1 size_t ConnectionTracker::cleanupStale(std::chrono::seconds timeout) {
2     auto now = std::chrono::steady_clock::now();
3     size_t removed = 0;
4
5     auto it = connections_.begin();
6     while (it != connections_.end()) {
7         auto age = std::chrono::duration_cast<std::chrono::seconds>(
8             now - it->second.last_seen);
9         if (age > timeout) {
10             it = connections_.erase(it);
11             removed++;
12         } else {
13             ++it;
14         }
15     }
16     return removed;
17 }

```

Listing 27: ]ConnectionTracker::cleanupStale[cite: 1]

The default timeout is 300 seconds (5 minutes), matching common NAT and firewall session expiry values.[cite: 1]

### 5.5.6 Per-Flow Statistics

Each `Connection` struct accumulates per-flow counters:[cite: 1]

```

1 struct Connection {
2     FiveTuple tuple;
3     ConnectionState state = ConnectionState::NEW;
4     AppType app_type = AppType::UNKNOWN;
5     std::string sni; // Extracted hostname (if any)
6
7     uint64_t packets_in = 0;
8     uint64_t packets_out = 0;
9     uint64_t bytes_in = 0;
10    uint64_t bytes_out = 0;
11
12    std::chrono::steady_clock::time_point first_seen;
13    std::chrono::steady_clock::time_point last_seen;
14
15    PacketAction action = PacketAction::FORWARD;
16
17    // TCP state tracking
18    bool syn_seen = false;
19    bool syn_ack_seen = false;
20    bool fin_seen = false;
21 };

```

Listing 28: ]Connection struct statistics fields (types.h)[cite: 1]

### 5.5.7 Global Connection Table

The `GlobalConnectionTable` aggregates statistics from all FP thread trackers for reporting purposes.[cite: 1] It holds raw pointers to each tracker and takes a shared read lock only when generating reports:[cite: 1]

```

1 GlobalConnectionTable::GlobalStats
2 GlobalConnectionTable::getGlobalStats() const {
3     std::shared_lock lock(mutex_);
4     GlobalStats stats;
5     for (auto* tracker : trackers_) {
6         auto ts = tracker->getStats();
7         stats.total_active_connections += ts.active_connections;
8         stats.total_connections_seen += ts.total_connections_seen;
9         // Aggregate app distribution
10        tracker->forEach([&](const Connection& conn) {
11            if (conn.app_type != AppType::UNKNOWN)
12                stats.app_distribution[conn.app_type]++;
13        });
14    }
15    return stats;
16 }

```

Listing 29: ]GlobalConnectionTable::getGlobalStats[cite: 1]

## 5.6 Load Balancer — src/load\_balancer.cpp

### 5.6.1 Responsibility

Each LoadBalancer thread acts as a **dispatcher**: it receives packets from the PCAP Reader, computes which Fast Path thread should handle each packet based on the five-tuple hash, and enqueues the packet into that FP’s input queue.[cite: 1]

### 5.6.2 Main Processing Loop

```

1 void LoadBalancer::run() {
2     while (running_) {
3         auto job_opt = input_queue_.popWithTimeout(
4             std::chrono::milliseconds(100));
5
6         if (!job_opt) continue; // Timeout, check running flag
7
8         packets_received_++;
9
10        // Consistent hashing: same tuple => same FP
11        int fp_idx = selectFP(job_opt->tuple);
12        fp_queues_[fp_idx]->push(std::move(*job_opt));
13
14        packets_dispatched_++;
15        per_fp_counts_[fp_idx]++;
16    }
17 }
18
19 int LoadBalancer::selectFP(const FiveTuple& tuple) {
20     FiveTupleHash hasher;
21     size_t hash = hasher(tuple);
22     return static_cast<int>(hash % num_fps_);
23 }

```

Listing 30: ]LoadBalancer::run — main dispatch loop[cite: 1]

**Why two tiers of load balancers?** The reader dispatches to LBs, and LBs dispatch to FPs.[cite: 1] This two-tier approach reduces lock contention on the shared FP queues: rather than the reader locking all FP queues directly, each LB independently manages a subset of FPs.[cite: 1]

### 5.6.3 LBManager: Routing the Reader to LBs

```

1 LoadBalancer& LBManager::getLBForPacket(const FiveTuple& tuple) {
2     FiveTupleHash hasher;
3     size_t hash = hasher(tuple);
4     int lb_idx = static_cast<int>(hash % lbs_.size());
5     return *lbs_[lb_idx];
6 }

```

Listing 31: ]LBManager::getLBForPacket[cite: 1]

Because both the LB selection and the FP selection use the same hash, the two-level routing is still deterministic — the same five-tuple always lands at the same FP.[cite: 1]

## 5.7 Fast Path Processor — src/fast\_path.cpp

### 5.7.1 Responsibility

The FastPathProcessor is the core DPI processing unit.[cite: 1] It owns:[cite: 1]

- An **input queue** (fed by an LB thread)[cite: 1]
- A **connection tracker** (per-thread, no sharing)[cite: 1]
- Access to the **rule manager** (shared, read-only)[cite: 1]
- An **output callback** (to forward or drop packets)[cite: 1]

### 5.7.2 Main Loop and Processing

```

1 void FastPathProcessor::run() {
2     while (running_) {
3         auto job_opt = input_queue_.popWithTimeout(
4             std::chrono::milliseconds(100));
5
6         if (!job_opt) {
7             // Idle: clean up stale connections
8             conn_tracker_.cleanupStale(std::chrono::seconds(300));
9             continue;
10        }
11
12        packets_processed_++;
13        PacketAction action = processPacket(*job_opt);
14        output_callback_(*job_opt, action);
15
16        if (action == PacketAction::DROP) packets_dropped_++;
17        else packets_forwarded_++;
18    }
19 }
20
21 PacketAction FastPathProcessor::processPacket(PacketJob& job) {
22     // Step 1: Get (or create) the connection entry
23     Connection* conn = conn_tracker_.getOrCreateConnection(job.tuple);
24     if (!conn) return PacketAction::FORWARD; // Failsafe
25
26     // Step 2: Update connection statistics
27     conn_tracker_.updateConnection(conn, job.data.size(), true);
28
29     // Step 3: Update TCP state machine
30     if (job.tuple.protocol == 6)
31         updateTCPState(conn, job.tcp_flags);
32
33     // Step 4: Fast path already blocked?
34     if (conn->state == ConnectionState::BLOCKED)
35         return PacketAction::DROP;
36
37     // Step 5: Payload inspection (only if not yet classified)
38     if (conn->state != ConnectionState::CLASSIFIED
39         && job.payload_length > 0)
40         inspectPayload(job, conn);
41 }

```



```

42 // Step 6: Rule matching
43 return checkRules(job, conn);
44 }

```

Listing 32: ]FastPathProcessor::run and processPacket[cite: 1]

### 5.7.3 Payload Inspection Logic

```

1 void FastPathProcessor::inspectPayload(PacketJob& job, Connection*
  conn) {
2     // Try TLS SNI first (HTTPS traffic      most common)
3     if (tryExtractSNI(job, conn)) return;
4
5     // Fall back to HTTP Host header (unencrypted traffic)
6     tryExtractHTTPHost(job, conn);
7 }
8
9 bool FastPathProcessor::tryExtractSNI(const PacketJob& job, Connection
  * conn) {
10     if (job.payload_offset >= job.data.size()) return false;
11
12     const uint8_t* payload = job.data.data() + job.payload_offset;
13     size_t payload_len     = job.payload_length;
14
15     auto sni = SNIExtractor::extract(payload, payload_len);
16     if (!sni) return false;
17
18     // Map SNI hostname to AppType
19     AppType app = sniToAppType(*sni);
20
21     // Mark connection as classified
22     conn_tracker_.classifyConnection(conn, app, *sni);
23     sni_extractions_++;
24     classification_hits_++;
25     return true;
26 }

```

Listing 33: ]FastPathProcessor::inspectPayload[cite: 1]

### 5.7.4 TCP State Tracking

```

1 void FastPathProcessor::updateTCPState(Connection* conn, uint8_t flags
  ) {
2     using namespace TCPFlags;
3
4     if (flags & SYN && !(flags & ACK)) {
5         conn->syn_seen = true;
6         conn->state     = ConnectionState::NEW;
7     }
8     else if (flags & SYN && flags & ACK) {
9         conn->syn_ack_seen = true;
10        if (conn->state == ConnectionState::NEW)
11            conn->state = ConnectionState::ESTABLISHED;
12    }
13    else if (flags & FIN || flags & RST) {
14        conn->fin_seen = true;

```

```

15     conn->state      = ConnectionState::CLOSED;
16 }
17 }

```

Listing 34: ]FastPathProcessor::updateTCPState[cite: 1]

### 5.7.5 Rule Checking

```

1 PacketAction FastPathProcessor::checkRules(
2     const PacketJob& job, Connection* conn) {
3
4     auto reason = rule_manager_->shouldBlock(
5         job.tuple.src_ip,
6         job.tuple.dst_port,
7         conn ? conn->app_type : AppType::UNKNOWN,
8         conn ? conn->sni      : "");
9
10    if (reason) {
11        // Block this packet and mark the entire connection
12        if (conn) conn_tracker_.blockConnection(conn);
13        return PacketAction::DROP;
14    }
15    return PacketAction::FORWARD;
16 }

```

Listing 35: ]FastPathProcessor::checkRules[cite: 1]

Notice that once a connection is marked **BLOCKED**, subsequent packets skip payload inspection and rule matching entirely (see `processPacket` Step 4) — this is the "fast path" optimisation that gives the component its name.[cite: 1]

## 5.8 DPI Engine — `src/dpi_engine.cpp`

### 5.8.1 Responsibility

`DPIEngine` is the **orchestrator**. [cite: 1] It owns all threads and components, manages their lifetimes, and coordinates startup and shutdown. [cite: 1]

### 5.8.2 Initialization Sequence

```

1 bool DPIEngine::initialize() {
2     // 1. Create the rule manager
3     rule_manager_ = std::make_unique<RuleManager>();
4     if (!config_.rules_file.empty())
5         rule_manager_->loadRules(config_.rules_file);
6
7     // 2. Define the output callback (called from FP threads)
8     auto output_cb = [this](const PacketJob& job, PacketAction action)
9     {
10         handleOutput(job, action);
11     };
12
13     // 3. Create FP Manager (spawns total_fps FP threads + queues)
14     int total_fps = config_.num_load_balancers * config_.fps_per_lb;
15     fp_manager_ = std::make_unique<FPManager>(

```

```

15     total_fps, rule_manager_.get(), output_cb);
16
17     // 4. Create LB Manager (spawns LB threads, wired to FP queues)
18     lb_manager_ = std::make_unique<LBManager>(
19         config_.num_load_balancers,
20         config_.fps_per_lb,
21         fp_manager_->getQueuePtrs());
22
23     // 5. Create global connection table (for reporting)
24     global_conn_table_ = std::make_unique<GlobalConnectionTable>(
25         total_fps);
26     for (int i = 0; i < total_fps; i++)
27         global_conn_table_->registerTracker(
28             i, &fp_manager_->getFP(i).getConnectionTracker());
29     return true;
30 }

```

Listing 36: ]DPIEngine::initialize — wiring all components together[cite: 1]

### 5.8.3 Startup and Shutdown Order

Order matters for clean startup and shutdown:[cite: 1]

| Phase           | Order               | Reason                                                   |
|-----------------|---------------------|----------------------------------------------------------|
| <b>Startup</b>  | Output Thread first | Must be ready to receive before producers start[cite: 1] |
|                 | FP Threads second   | Must be ready before LBs enqueue[cite: 1]                |
|                 | LB Threads third    | Must be ready before Reader starts[cite: 1]              |
|                 | Reader Thread last  | No packets until pipeline is ready[cite: 1]              |
| <b>Shutdown</b> | LBs first           | Stop feeding FPs; drain LB queues[cite: 1]               |
|                 | FPs second          | Drain FP queues; stop producing output[cite: 1]          |
|                 | Output thread last  | Drain output queue; finish writing[cite: 1]              |

### 5.8.4 Reader Thread

```

1 void DPIEngine::readerThreadFunc(const std::string& input_file) {
2     PacketAnalyzer::PcapReader reader;
3     if (!reader.open(input_file)) return;
4
5     PacketAnalyzer::RawPacket raw;
6     PacketAnalyzer::ParsedPacket parsed;
7     uint32_t packet_id = 0;
8
9     while (reader.readNextPacket(raw) && running_) {
10         if (PacketParser::parse(raw, parsed)) {
11             // Build a PacketJob (adds DPI metadata)
12             PacketJob job = createPacketJob(raw, parsed, ++packet_id);
13
14             // Route to the correct LB via five-tuple hash
15             LoadBalancer& lb = lb_manager_->getLBForPacket(job.tuple);
16             lb.getInputQueue().push(std::move(job));
17         }
18     }
19 }

```

```

18         stats_.total_packets++;
19         stats_.total_bytes += raw.captured_length;
20     }
21 }
22
23 processing_complete_ = true;
24 }

```

Listing 37: ]DPIEngine::readerThreadFunc[cite: 1]

### 5.8.5 Output Handling

```

1 void DPIEngine::handleOutput(const PacketJob& job, PacketAction action
  ) {
2     stats_.forwarded_packets++;
3     if (action != PacketAction::DROP) {
4         output_queue_.push(job); // Thread-safe, may block if queue
                                   full
5     } else {
6         stats_.dropped_packets++;
7     }
8 }
9
10 void DPIEngine::outputThreadFunc() {
11     while (running_ || !output_queue_.empty()) {
12         auto job_opt = output_queue_.popWithTimeout(
13             std::chrono::milliseconds(50));
14         if (job_opt) writeOutputPacket(*job_opt);
15     }
16 }

```

Listing 38: ]DPIEngine::handleOutput and outputThreadFunc[cite: 1]

### 5.8.6 Statistics and Reporting

The DPIStats struct uses `std::atomic` counters to allow lock-free updates from multiple FP threads simultaneously:[cite: 1]

```

1 struct DPIStats {
2     std::atomic<uint64_t> total_packets{0};
3     std::atomic<uint64_t> total_bytes{0};
4     std::atomic<uint64_t> forwarded_packets{0};
5     std::atomic<uint64_t> dropped_packets{0};
6     std::atomic<uint64_t> tcp_packets{0};
7     std::atomic<uint64_t> udp_packets{0};
8     std::atomic<uint64_t> active_connections{0};
9
10    // Non-copyable (atomics are not copyable)
11    DPIStats() = default;
12    DPIStats(const DPIStats&) = delete;
13    DPIStats& operator=(const DPIStats&) = delete;
14 };

```

Listing 39: ]DPIStats atomic counters (types.h)[cite: 1]

Atomic operations on modern x86-64 CPUs are lock-free for 64-bit integers, making counter updates from multiple threads safe without any mutex overhead.[cite: 1]

## 6 Data Flow: Tracing One Packet End-to-End

This section traces a single HTTPS packet (a TLS Client Hello to `www.youtube.com`) through the entire multi-threaded pipeline.[cite: 1]

1. PcapReader reads 16-byte header + raw Ethernet frame bytes
2. PacketParser decodes Ethernet → IPv4 → TCP headers; finds payload
3. Reader computes `FiveTuple{192.168.1.5, 142.250.10.46, 52341, 443, TCP}`
4. `hash(tuple) % 2 = 0` → pushed to LB0's input queue
5. LB0 pops packet; `hash(tuple) % 2 = 1` → pushed to FP1's queue
6. FP1 pops packet; calls `getOrCreateConnection(tuple)` → new entry
7. TCP flags = SYN+ACK → state set to ESTABLISHED
8. `SNIExtractor::isTLSClientHello()` → true
9. `SNIExtractor::extract()` parses extensions → "www.youtube.com"
10. `sniToAppType("www.youtube.com")` → `AppType::YOUTUBE`
11. `classifyConnection(conn, YOUTUBE, "www.youtube.com")` → state = CLASSIFIED
12. `isAppBlocked(YOUTUBE)` → true (rule active) → DROP
13. Output callback called with action=DROP → `stats.dropped_packets++`

Figure 13: Step-by-step packet trace: HTTPS to YouTube (blocked)[cite: 1]

All subsequent packets from the same five-tuple are dropped at **Step 12** without repeating SNI extraction — the connection is permanently marked BLOCKED.[cite: 1]

## 7 Build System and Compilation

### 7.1 CMake Configuration

The project uses CMake (`CMakeLists.txt`) for cross-platform builds:[cite: 1]

Listing 40: ]Building the project on Linux/macOS[cite: 1]

```
# Create build directory
mkdir build && cd build

# Configure with CMake
cmake .. -DCMAKE_BUILD_TYPE=Release

# Compile (use all CPU cores)
make -j$(nproc)
```

Listing 41: ]Generated executables[cite: 1]

```
./packet_analyzer      # Simple single-threaded version
./dpi_engine           # Full multi-threaded DPI engine
```

## 7.2 Compiler Requirements

| Requirement         | Details                                                                                                   |
|---------------------|-----------------------------------------------------------------------------------------------------------|
| <b>C++ Standard</b> | C++17 (required for <code>std::optional</code> , <code>std::shared_mutex</code> , structured bindings)[ci |
| <b>Compiler</b>     | GCC 8+ or Clang 7+[cite: 1]                                                                               |
| <b>CMake</b>        | Version 3.14+[cite: 1]                                                                                    |
| <b>OS</b>           | Linux, macOS, Windows (with MinGW or MSVC + WINDOWS_SETUP.md)[ci                                          |
| <b>Dependencies</b> | None (no libpcap required — PCAP parsing is implemented natively)[cite: 1]                                |

## 7.3 Generating Test Traffic

The included Python script generates a synthetic PCAP file with various traffic types:[cite: 1]

Listing 42: ]Generating test PCAP[cite: 1]

```
# Requires: pip install scapy
python3 generate_test_pcap.py

# Output: test_dpi.pcap containing:
# - TLS Client Hello packets to YouTube, Facebook, Netflix
# - HTTP requests with Host headers
# - DNS queries
# - Mixed TCP and UDP flows
```

## 8 Running the Engine and Interpreting Output

### 8.1 Simple Mode

Listing 43: ]Running the simple packet analyzer[cite: 1]

```
# Print all packets
./packet_analyzer capture.pcap

# Print first 20 packets only
./packet_analyzer capture.pcap 20
```

Sample output:

```
===== Packet #1 =====
Time: 2024-01-15 10:23:45.123456

[Ethernet]
  Source MAC:      AA:BB:CC:DD:EE:FF
  Destination MAC: 11:22:33:44:55:66
  EtherType:      0x0800 (IPv4)

[IPv4]
  Source IP:       192.168.1.100
  Destination IP:  142.250.10.46
  Protocol:        TCP
  TTL:             64
```

```
[TCP]
  Source Port:      54321
  Destination Port: 443
  Sequence Number:  1234567890
  Ack Number:       0
  Flags:            SYN

[Payload]
  Length: 517 bytes
  Preview: 16 03 01 02 00 01 00 01 fc 03 03 ...
```

## 8.2 DPI Multi-Threaded Mode

Listing 44: ]Running the DPI engine[cite: 1]

```
# Basic usage (uses default 2 LBs, 4 FP threads)
./dpi_engine test_dpi.pcap output.pcap

# With rules file
./dpi_engine test_dpi.pcap output.pcap --rules rules.txt

# With custom thread counts
./dpi_engine test_dpi.pcap output.pcap --lb 4 --fp 8
```

### Sample output:

```
DPI ENGINE v1.0
Configuration: Load Balancers=2, FPs per LB=2, Total FP Threads=4

[DPIEngine] Initialized successfully
[FP0] Started
[FP1] Started
[FP2] Started
[FP3] Started
[LB0] Started (serving FP0-FP1)
[LB1] Started (serving FP2-FP3)

Processing complete.

=== STATISTICS ===
  Total packets:      15,234
  Forwarded:          12,891
  Dropped:             2,343
  TCP flows:           8,401
  UDP flows:           3,211
  Active connections:  142

=== APP DISTRIBUTION ===
  YOUTUBE:      3,211 flows
  NETFLIX:      1,854 flows
  FACEBOOK:     1,203 flows [BLOCKED]
  GOOGLE:       2,133 flows
  UNKNOWN:      ...
```

## 8.3 Rule File Management

Listing 45: ]Example rules file with comments[cite: 1]

```
# Block social media
block_app FACEBOOK
block_app INSTAGRAM
block_app TIKTOK
block_domain *.tiktok.com

# Block specific IPs (known bad actors)
block_ip 203.0.113.42

# Block BitTorrent ports
block_port 6881
block_port 6882

# Block streaming during work hours
block_app YOUTUBE
block_app NETFLIX
block_app SPOTIFY
```

## 9 Web Deployment Architecture

The project was extended beyond its original command-line form into a **full-stack web application** that allows any user to upload a `.pcap` file through a browser, run the DPI engine on it, and visualize results — all without installing any local tooling.[cite: 1]

### 9.1 Deployment Overview

The web deployment follows a **split-hosting** architecture:[cite: 1]

| Component              | Host             | Technology                                |
|------------------------|------------------|-------------------------------------------|
| <b>React Frontend</b>  | Vercel (CDN)     | Vite + Recharts[cite: 1]                  |
| <b>FastAPI Backend</b> | Render (Docker)  | Python 3.11 + uvicorn[cite: 1]            |
| <b>C++ DPI Binary</b>  | Render container | Compiled at build time via CMake[cite: 1] |

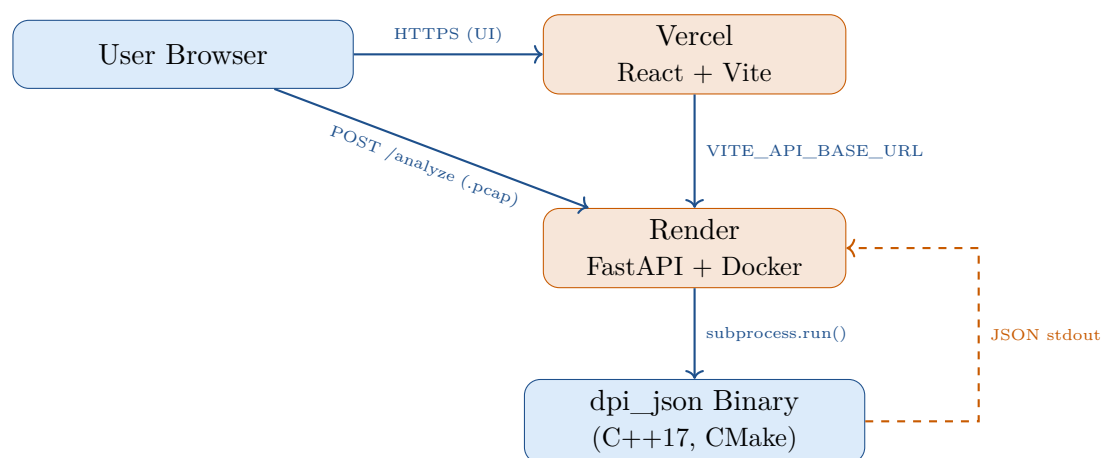


Figure 14: Web deployment — split frontend/backend architecture[cite: 1]



## 9.2 Monorepo Folder Structure

The repository is organized as a **monorepo** with three top-level directories:[cite: 1]

Listing 46: ]Repository layout after web deployment addition[cite: 1]

```
DPI_Engine/
  packet-analyzer-core/  # Original C++ standalone engine
    include/
    src/
    CMakeLists.txt

  backend-render/        # Render deployment target
    include/             # Same C++ headers (symlinked/
copied)
    src/
      main_json.cpp      # Web-optimised JSON output
entry point
      server.py           # FastAPI application
      requirements.txt
      render.yaml         # Render service definition
      CMakeLists.txt

  frontend-vercel/       # Vercel deployment target
    src/
      App.jsx             # Main React component
      main.jsx
      index.html
      package.json
      vite.config.js
      vercel.json
```

## 9.3 render.yaml — Infrastructure as Code

Render’s deployment is fully specified in `backend-render/render.yaml`:[cite: 1]

Listing 47: ]render.yaml — Render service definition[cite: 1]

```
services:
- type: web
  name: dpi-engine-api
  env: python
  plan: free
  rootDir: backend-render
  buildCommand: |
    pip install -r requirements.txt
    cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
    cmake --build build --config Release
  startCommand: uvicorn server:app --host 0.0.0.0 --port $PORT
  envVars:
    - key: FRONTEND_ORIGIN
  sync: false
```

Key design decisions:

- `buildCommand` compiles the C++ binary *inside* the Render build environment — no pre-compiled binary is committed to git.[cite: 1]

- `cmake -build build` produces `build/dpi_json`, which `server.py` invokes via `subprocess.run()` [1]
- `FRONTEND_ORIGIN` is an environment variable set in the Render dashboard — it configures CORS to allow only the deployed Vercel URL, tightening security beyond the development wildcard `"*"`. [cite: 1]

## 9.4 FastAPI Backend — `backend-render/server.py`

### 9.4.1 Responsibility

The FastAPI server acts as a **bridge** between the browser and the compiled C++ binary. [cite: 1] It handles: [cite: 1]

1. File upload validation and storage [cite: 1]
2. Constructing the binary command with blocking rules [cite: 1]
3. Invoking the binary via `subprocess.run()` with a 60-second timeout [cite: 1]
4. Parsing and returning the JSON output [cite: 1]
5. Serving the filtered output PCAP for download [cite: 1]

### 9.4.2 CORS Configuration

```

1 FRONTEND_ORIGIN = os.getenv("FRONTEND_ORIGIN", "*")
2
3 app.add_middleware(
4     CORSMiddleware,
5     allow_origins=["*"] if FRONTEND_ORIGIN == "*"
6     else [o.strip() for o in FRONTEND_ORIGIN.split(",")],
7     allow_methods=["*"],
8     allow_headers=["*"],
9 )

```

Listing 48: ]Environment-variable-driven CORS (`server.py`) [cite: 1]

In development (`FRONTEND_ORIGIN` unset), all origins are permitted. [cite: 1] In production, Render's environment variable restricts access to the Vercel URL. [cite: 1]

### 9.4.3 File Upload and Binary Invocation

```

1 @app.post("/analyze")
2 async def analyze(
3     file: UploadFile = File(...),
4     block_apps: Optional[str] = Form(default=""),
5 ):
6     if not file.filename.endswith(".pcap"):
7         raise HTTPException(400, "Only .pcap files are supported")
8
9     job_id      = str(uuid.uuid4())[:8]
10    input_path   = UPLOAD_DIR / f"{job_id}_input.pcap"
11    output_path  = OUTPUT_DIR / f"{job_id}_output.pcap"
12

```

```

13     contents = await file.read()
14     input_path.write_bytes(contents)
15
16     # Build CLI command with optional blocking rules
17     cmd = [str(BINARY), str(input_path), str(output_path)]
18     if block_apps:
19         for app_name in block_apps.split(","):
20             app_name = app_name.strip().upper()
21             if app_name in KNOWN_APPS:
22                 cmd += ["--block-app", app_name]
23
24     result = subprocess.run(cmd, capture_output=True, text=True,
25                             timeout=60)
26
27     if result.returncode != 0:
28         raise HTTPException(500, f"Analysis failed: {result.stderr
29                                     [:200]}")
30
31     analysis = json.loads(result.stdout)
32     analysis["job_id"] = job_id
33     analysis["filename"] = file.filename
34     analysis["blocked_apps"] = [a.strip().upper()
35                                 for a in (block_apps or "").split(",")
36                                 if a.strip()]
37
38     return JSONResponse(analysis)

```

Listing 49: ]analyze endpoint — upload to binary invocation (server.py)[cite: 1]

#### 9.4.4 API Endpoints

| Method | Path               | Description                                       |
|--------|--------------------|---------------------------------------------------|
| GET    | /health            | Binary found check; uptime probe[cite: 1]         |
| POST   | /analyze           | Upload .pcap, returns full JSON analysis[cite: 1] |
| GET    | /download/{job_id} | Download filtered output PCAP[cite: 1]            |
| GET    | /apps              | List of blockable application names[cite: 1]      |

## 9.5 main\_json.cpp — Web-Optimised C++ Entry Point

### 9.5.1 Responsibility

src/main\_json.cpp is a new entry point added specifically for the web deployment.[cite: 1] Unlike main.cpp (which prints human-readable text to stdout) or dpi\_mt.cpp (which runs the full multi-threaded engine), this binary:[cite: 1]

- Accepts optional `--block-app APP_NAME` arguments[cite: 1]
- Runs packet analysis in a single thread (suitable for per-request cloud invocation)[cite: 1]
- Outputs a **single JSON object** to stdout, which FastAPI reads directly[cite: 1]
- Writes allowed packets to an output PCAP file for download[cite: 1]

### 9.5.2 JSON Output Schema

Listing 50: JSON output structure of dpi\_json[cite: 1]

```
{
  "summary": {
    "total_packets": 15234,
    "forwarded": 12891,
    "dropped": 2343,
    "total_flows": 842,
    "tcp_packets": 11201,
    "udp_packets": 4033
  },
  "app_distribution": [
    { "app": "YOUTUBE", "count": 3211, "percent": 21.1 },
    { "app": "GOOGLE", "count": 2133, "percent": 14.0 },
    ...
  ],
  "detected_domains": [
    { "domain": "www.youtube.com", "app": "YOUTUBE" },
    { "domain": "api.facebook.com", "app": "FACEBOOK" },
    ...
  ],
  "top_flows": [
    { "src": "192.168.1.5:52341", "dst": "142.250.10.46:443",
      "proto": "TCP", "app": "YOUTUBE", "sni": "www.youtube.com",
      "packets": 412, "bytes": 589200, "blocked": true },
    ...
  ],
  "packets": [
    { "num": 1, "time": "10:23:45.123", "src": "192.168.1.5:52341",
      "dst": "142.250.10.46:443", "proto": "TCP", "app": "YOUTUBE",
      "sni": "www.youtube.com", "size": 571, "blocked": true,
      "flags": "SYN" },
    ...
  ]
}
```

### 9.5.3 Application Detection Logic in main\_json.cpp

The binary combines three detection methods in priority order:[cite: 1]

```
1 // 1. TLS SNI (HTTPS, port 443)
2 if (dest_port == 443 && payload_available) {
3     auto sni = SNIExtractor::extract(payload, payload_len);
4     if (sni) { flow.sni = *sni; flow.app_type = sniToAppType(*sni); }
5 }
6
7 // 2. HTTP Host header (HTTP, port 80)
8 if (dest_port == 80 && payload_available && flow.sni.empty()) {
9     auto host = HTTPHostExtractor::extract(payload, payload_len);
10    if (host) { flow.sni = *host; flow.app_type = sniToAppType(*host); }
11 }
12
13 // 3. Port-based fallback (when payload inspection yields nothing)
14 if (flow.app_type == AppType::UNKNOWN) {
```

```

15     if      (dest_port == 443 || src_port == 443) flow.app_type =
        AppType::HTTPS;
16     else if (dest_port == 80  || src_port == 80 ) flow.app_type =
        AppType::HTTP;
17     else if (dest_port == 53  || src_port == 53 ) flow.app_type =
        AppType::DNS;
18 }

```

Listing 51: ]Three-tier detection in main\_json.cpp[cite: 1]

### 9.5.4 JSON String Escaping

Because the output is hand-built (no external JSON library), a helper function correctly escapes special characters to produce valid JSON:[cite: 1]

```

1 std::string jsonEscape(const std::string& s) {
2     std::string out;
3     for (char c : s) {
4         if      (c == '\"') out += "\\\"";
5         else if (c == '\\') out += "\\\\";
6         else if (c == '\n') out += "\\n";
7         else if (c == '\r') out += "\\r";
8         else      out += c;
9     }
10    return out;
11 }

```

Listing 52: ]jsonEscape helper (main\_json.cpp)[cite: 1]

This is necessary because SNI hostnames and IP addresses extracted from network traffic may contain characters that would break JSON parsing if not escaped.[cite: 1]

## 9.6 React Frontend — frontend-vercel/src/App.jsx

### 9.6.1 Responsibility

The React frontend provides a browser-based interface with four distinct views and communicates exclusively with the FastAPI backend via the VITE\_API\_BASE\_URL environment variable:[cite: 1]

```

1 // Reads from Vercel environment variable at build time;
2 // falls back to localhost for local development
3 const API = import.meta.env.VITE_API_BASE_URL || "http://localhost
        :8000";

```

Listing 53: ]Environment-variable API routing (App.jsx)[cite: 1]

### 9.6.2 Component Architecture

The application is structured as a single-file React component with four logical sub-components:[cite: 1]

| Component          | Responsibility                                                                                                     |
|--------------------|--------------------------------------------------------------------------------------------------------------------|
| <b>DropZone</b>    | Drag-and-drop / click-to-browse file upload; shows animated progress bar during analysis.[cite: 1]                 |
| <b>AppPill</b>     | Toggle button for pre-analysis blocking rules (e.g. click YOUTUBE to block before uploading).[cite: 1]             |
| <b>StatCard</b>    | Displays a single summary metric (total packets, forwarded, dropped, flows).[cite: 1]                              |
| <b>PacketTable</b> | Paginated, filterable table of up to 500 captured packets; supports filtering by IP, app name, or domain.[cite: 1] |

### 9.6.3 Four Dashboard Tabs

| Tab             | Icon | Content                                                                                                     |
|-----------------|------|-------------------------------------------------------------------------------------------------------------|
| <b>Overview</b> |      | Recharts PieChart of app distribution + horizontal BarChart of top 8 apps by packet count[cite: 1]          |
| <b>Packets</b>  |      | Full PacketTable with per-packet SNI, flags, size, and BLOCKED/OK badge[cite: 1]                            |
| <b>Flows</b>    |      | Top 50 flows sorted by bytes transferred; shows src:port, dst:port, protocol, app, SNI, and status[cite: 1] |
| <b>Domains</b>  |      | Card grid of all unique SNI hostnames detected, coloured by application type[cite: 1]                       |

### 9.6.4 File Upload and Analysis Flow

```

1  const analyze = useCallback(async (f) => {
2    setLoading(true);
3
4    const fd = new FormData();
5    fd.append("file", f);
6    fd.append("block_apps", [...blocked].join(",")); // e.g. "YOUTUBE
7      ,TIKTOK"
8
9    const res = await fetch(`${API}/analyze`, { method: "POST", body:
10      fd });
11    const json = await res.json();
12
13    setData(json); // triggers re-render with results
14    setTab("overview");
15  }, [blocked]);

```

Listing 54: `]analyze()` function in `App.jsx` — multipart upload to backend[cite: 1]

The `FormData` object carries both the binary PCAP file and the comma-separated list of apps to block in a single `multipart/form-data` POST request.[cite: 1]

### 9.6.5 Vite Configuration and Vercel Deployment

Listing 55: ]vercel.json — single-page app routing[cite: 1]

```
{
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

Listing 56: ]vite.config.js — dev proxy for local development[cite: 1]

```
server: {
  port: 3000,
  proxy: {
    "/analyze": "http://localhost:8000",
    "/download": "http://localhost:8000",
    "/apps":    "http://localhost:8000",
    "/health":  "http://localhost:8000",
  },
},
```

The Vite dev proxy means that during local development the frontend runs on port 3000 and transparently forwards API calls to the FastAPI server on port 8000 — no CORS issues in development.[cite: 1]

## 9.7 Docker Multi-Stage Build

The Dockerfile uses a **three-stage build** to produce a minimal final image:[cite: 1]

Listing 57: ]Three-stage Dockerfile overview[cite: 1]

```
# Stage 1: Compile C++ binary (ubuntu:22.04 + cmake + g++)
FROM ubuntu:22.04 AS cpp-builder
RUN apt-get install -y cmake g++ make
COPY source/ source/
COPY include/ include/
COPY CMakeLists.txt .
RUN mkdir build && cmake .. && make -j$(nproc) dpi_json

# Stage 2: Build React static files (node:20-alpine)
FROM node:20-alpine AS frontend-builder
COPY frontend/package.json ./
RUN npm install
COPY frontend/ .
RUN npm run build          # outputs to dist/

# Stage 3: Minimal runtime (python:3.11-slim)
FROM python:3.11-slim
COPY --from=cpp-builder /app/build/dpi_json build/dpi_json
COPY --from=frontend-builder /app/frontend/dist frontend/dist
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY server.py .
CMD ["uvicorn", "server:app", "--host", "0.0.0.0", "--port", "8000"]
```

### Why multi-stage?

- The final image does *not* contain `cmake`, `g++`, `node`, or `npm` — only the compiled binary and the Python runtime.[cite: 1] This reduces the image size from  $\approx 1.8$  GB (if all tools were kept) to  $\approx 180$  MB.[cite: 1]

- Each stage is cached independently by Docker — a change to only the frontend does not re-trigger a C++ compilation.[cite: 1]

## 9.8 End-to-End Web Request Lifecycle

Tracing a complete analysis request from browser to response:[cite: 1]

1. User selects `capture.pcap` in browser; toggles YOUTUBE blocked
2. `fetch(API+"/analyze")` sends `multipart/form-data` POST
3. FastAPI `/analyze`: validates extension, writes to `uploads/abc12345_input.pcap`
4. Builds command: `./build/dpi_json uploads/... outputs/... -block-app YOUTUBE`
5. `subprocess.run()` invokes C++ binary (timeout 60 s)
6. Binary reads PCAP, extracts SNI/Host, classifies flows, applies rules
7. Binary writes filtered `output.pcap`; prints JSON to `stdout`
8. FastAPI parses JSON, attaches `job_id/filename/blocked_apps`
9. `JSONResponse` returned to browser over HTTPS
10. React renders: 6 stat cards, pie chart, packet table, domain cards
11. User clicks “Download Filtered PCAP” → `GET /download/abc12345`

Figure 15: Complete web request lifecycle — browser to response[cite: 1]



## 9.9 Security Considerations in Web Deployment

| Concern                | Mitigation                                                                                                                                       |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Malicious file uploads | Extension check (.pcap only); 50 MB size limit; binary runs in a sandboxed subprocess.[cite: 1]                                                  |
| Path traversal         | job_id is UUID-derived; download endpoint validates that the ID is alphanumeric only and under 40 characters.[cite: 1]                           |
| CORS                   | FRONTEND_ORIGIN env var restricts allowed origins in production; wildcard only used in development.[cite: 1]                                     |
| Subprocess injection   | Binary path is hardcoded; user-supplied app names are validated against a fixed KNOWN_APPS whitelist before being added to the command.[cite: 1] |
| Timeout DoS            | subprocess.run(timeout=60) kills the binary if it exceeds 60 seconds; prevents resource exhaustion on large files.[cite: 1]                      |
| Disk cleanup           | Input PCAP is deleted in the finally block after each request regardless of success or failure.[cite: 1]                                         |

## 10 Comparison: CLI vs. Web Deployment Modes

| Aspect          | CLI / Standalone                  | Web Deployment                         |
|-----------------|-----------------------------------|----------------------------------------|
| Entry Point     | main.cpp, dpi_mt.cpp              | main_json.cpp[cite: 1]                 |
| Threading Model | Multi-threaded (LB + FP pipeline) | Single-threaded per request[cite: 1]   |
| Output Format   | Human-readable terminal text      | Structured JSON to stdout[cite: 1]     |
| User Interface  | Terminal                          | Browser (React + Recharts)[cite: 1]    |
| Blocking Rules  | rules.txt file                    | HTTP form field (block_apps)[cite: 1]  |
| Deployment      | Local compile + run               | Docker on Render + Vercel CDN[cite: 1] |
| Suitable For    | Large captures, batch processing  | Interactive analysis, demos[cite: 1]   |

## 11 Performance Considerations

### 11.1 Zero-Copy Payload Inspection

The payload pointer in `ParsedPacket` and `PacketJob` points directly into the original byte vector — no additional memory allocation is needed for payload inspection.[cite: 1]  
This is significant because:[cite: 1]

- Memory allocation (even from the OS allocator) can take hundreds of nanoseconds per call.[cite: 1]
- TLS Client Hello payloads are typically 300–600 bytes; avoiding a copy saves memory bandwidth.[cite: 1]

## 11.2 Lock-Free Counters

Global statistics use `std::atomic<uint64_t>` which maps to a single `lock xadd` instruction on x86-64 — effectively free in terms of latency compared to mutex-based synchronisation.[cite: 1]

## 11.3 Per-Thread Connection Tables

Each FP thread maintains its own `unordered_map` for connection tracking.[cite: 1] This design eliminates **false sharing** (different threads writing to the same cache line) and avoids lock contention on the connection table entirely.[cite: 1]

## 11.4 Bounded Queues and Backpressure

Each inter-thread queue is bounded at 10,000 items.[cite: 1] When a queue fills up, the producing thread blocks.[cite: 1] This implements automatic **backpressure**: if the FP threads are slower than the reader, the reader naturally slows down rather than consuming unbounded memory.[cite: 1]

## 11.5 Scalability

The engine scales linearly with the number of FP threads up to the number of physical CPU cores (after which context-switching overhead dominates).[cite: 1] Recommended configuration:[cite: 1]

| Traffic Volume     | LB Threads | FP Threads | Expected Throughput |
|--------------------|------------|------------|---------------------|
| Small (< 1 Gbps)   | 1          | 2          | Adequate[cite: 1]   |
| Medium (1–10 Gbps) | 2          | 4          | Good[cite: 1]       |
| Large (> 10 Gbps)  | 4          | 8–16       | High[cite: 1]       |

# 12 Limitations and Future Enhancements

## 12.1 Current Limitations

1. **File-based only**: The engine reads from PCAP files.[cite: 1] Real network DPI would use a raw socket or DPDK (Data Plane Development Kit) to capture live traffic.[cite: 1]
2. **TLS 1.3 ESNI/ECH**: TLS 1.3 introduced Encrypted Client Hello (ECH), which encrypts the SNI.[cite: 1] This engine cannot extract the SNI from ECH-protected connections.[cite: 1]

3. **IPv6:** The parser handles IPv4 only.[cite: 1] IPv6 headers have a different format and require separate parsing logic.[cite: 1]
4. **IP Fragmentation:** Fragmented IP datagrams are not reassembled.[cite: 1] A TLS Client Hello spread across two fragments would not be detected.[cite: 1]
5. **TCP Reassembly:** The engine inspects individual packets, not full TCP streams.[cite: 1] An application-layer message split across multiple TCP segments may not be classified correctly.[cite: 1]

## 12.2 Potential Enhancements

- **Live capture:** Replace `PcapReader` with a `libpcap` or `AF_PACKET` socket for real-time processing.[cite: 1]
- **DPDK integration:** Use Intel DPDK for kernel-bypass packet capture, achieving multi-10Gbps throughput.[cite: 1]
- **Machine learning classification:** For traffic that cannot be identified by SNI (e.g. ESNI, non-TLS protocols), train a classifier on packet timing and size patterns.[cite: 1]
- **REST API:** Expose the rule manager through an HTTP API for dynamic rule updates without restarting the engine.[cite: 1]
- **Prometheus metrics:** Export statistics in Prometheus format for integration with Grafana dashboards.[cite: 1]
- **IPv6 support:** Add an `IPv6Header` struct and parsing logic.[cite: 1]

## 13 Glossary

| Term                | Definition                                                                                       |
|---------------------|--------------------------------------------------------------------------------------------------|
| <b>ACK</b>          | TCP acknowledgment flag; indicates the acknowledgment field is valid.[cite: 1]                   |
| <b>ARP</b>          | Address Resolution Protocol; maps IP addresses to MAC addresses on a LAN.[cite: 1]               |
| <b>Big-Endian</b>   | Byte order where the most significant byte is stored first (used in network protocols).[cite: 1] |
| <b>Cipher Suite</b> | A named combination of cryptographic algorithms used in a TLS session.[cite: 1]                  |
| <b>Client Hello</b> | The first TLS handshake message, sent by the client to initiate a secure connection.[cite: 1]    |
| <b>Connection</b>   | A stateful network flow identified by a five-tuple.[cite: 1]                                     |
| <b>DPI</b>          | Deep Packet Inspection; analysis of packet payloads beyond header information.[cite: 1]          |

| Term               | Definition                                                                                               |
|--------------------|----------------------------------------------------------------------------------------------------------|
| <b>DPDK</b>        | Data Plane Development Kit; Intel library for kernel-bypass networking.[cite: 1]                         |
| <b>ECH</b>         | Encrypted Client Hello; TLS extension that encrypts the SNI.[cite: 1]                                    |
| <b>Ethernet</b>    | The Layer 2 protocol used in most wired and wireless LANs.[cite: 1]                                      |
| <b>Fast Path</b>   | Processing path optimised for speed, typically skipping redundant checks.[cite: 1]                       |
| <b>FIN</b>         | TCP flag indicating the sender has finished sending data (connection closing).[cite: 1]                  |
| <b>Five-Tuple</b>  | The combination (src IP, dst IP, src port, dst port, protocol) that uniquely identifies a flow.[cite: 1] |
| <b>Flow</b>        | A sequence of packets sharing the same five-tuple.[cite: 1]                                              |
| <b>Host Header</b> | HTTP/1.1 header specifying the domain name of the requested resource.[cite: 1]                           |
| <b>HTTPS</b>       | HTTP over TLS; encrypted web traffic, typically on port 443.[cite: 1]                                    |
| <b>IHL</b>         | IP Header Length; lower 4 bits of the first IPv4 byte, in 32-bit words.[cite: 1]                         |
| <b>IPv4</b>        | Internet Protocol version 4; uses 32-bit addresses.[cite: 1]                                             |
| <b>LB</b>          | Load Balancer; thread that distributes packets across FP threads.[cite: 1]                               |
| <b>MAC Address</b> | 6-byte hardware identifier assigned to a network interface.[cite: 1]                                     |
| <b>PCAP</b>        | Packet Capture; standard binary file format for stored network traffic.[cite: 1]                         |
| <b>PSH</b>         | TCP push flag; instructs the receiver to pass data to the application immediately.[cite: 1]              |
| <b>QUIC</b>        | Quick UDP Internet Connections; transport protocol for HTTP/3, runs over UDP.[cite: 1]                   |
| <b>RST</b>         | TCP reset flag; abruptly terminates a connection.[cite: 1]                                               |
| <b>SNI</b>         | Server Name Indication; TLS extension carrying the target hostname in plaintext.[cite: 1]                |
| <b>SYN</b>         | TCP synchronise flag; used to initiate a connection.[cite: 1]                                            |
| <b>TCP</b>         | Transmission Control Protocol; reliable, ordered, connection-oriented Layer 4 protocol.[cite: 1]         |
| <b>TLS</b>         | Transport Layer Security; cryptographic protocol providing encrypted communication.[cite: 1]             |
| <b>TTL</b>         | Time To Live; IPv4 field decremented at each router hop.[cite: 1]                                        |
| <b>UDP</b>         | User Datagram Protocol; connectionless Layer 4 protocol with no delivery guarantees.[cite: 1]            |

## 14 Conclusion

---

This document has presented a comprehensive study of the DPI Engine project, covering all layers from fundamental networking theory through to multi-threaded software architecture.[cite: 1]

### Networking theory covered:

- The OSI and TCP/IP layered models[cite: 1]
- Ethernet frame, IPv4 datagram, TCP segment, and UDP datagram structures[cite: 1]
- The five-tuple as a flow identifier[cite: 1]
- The PCAP file format and its binary layout[cite: 1]
- TLS handshake mechanics and SNI extraction[cite: 1]
- HTTP Host header and DNS query analysis[cite: 1]
- Application classification strategies[cite: 1]

### Software architecture covered:

- **PcapReader** — binary file I/O and global/packet header parsing[cite: 1]
- **PacketParser** — zero-copy, layered protocol decoding[cite: 1]
- **SNIExtractor** — byte-level TLS Client Hello navigation[cite: 1]
- **RuleManager** — reader-writer lock protected rule storage[cite: 1]
- **ConnectionTracker** — per-thread flow state with timeout eviction[cite: 1]
- **LoadBalancer** — consistent-hash based packet dispatch[cite: 1]
- **FastPathProcessor** — stateful payload inspection and rule matching[cite: 1]
- **DPIEngine** — thread lifecycle management and output coordination[cite: 1]
- **ThreadSafeQueue** — bounded, blocking inter-thread communication[cite: 1]

The DPI Engine demonstrates how modern C++ features — move semantics, `std::optional`, `std::shared_mutex`, `std::atomic`, and template-based generic queues — combine with classical networking algorithms (consistent hashing, TCP state machines, protocol parsing) to build a production-grade packet processing system.[cite: 1]

## References

---

- [1] J. Postel, *Transmission Control Protocol*, RFC 793, IETF, September 1981. <https://www.rfc-editor.org/rfc/rfc793>
- [2] J. Postel, *Internet Protocol*, RFC 791, IETF, September 1981. <https://www.rfc-editor.org/rfc/rfc791>
- [3] E. Rescorla, *The Transport Layer Security (TLS) Protocol Version 1.3*, RFC 8446, IETF, August 2018. <https://www.rfc-editor.org/rfc/rfc8446>
- [4] D. Eastlake, *Transport Layer Security (TLS) Extensions: Extension Definitions (SNI)*, RFC 6066, IETF, January 2011. <https://www.rfc-editor.org/rfc/rfc6066>
- [5] J. Iyengar and M. Thomson, *QUIC: A UDP-Based Multiplexed and Secure Transport*, RFC 9000, IETF, May 2021. <https://www.rfc-editor.org/rfc/rfc9000>
- [6] The Tcpdump Group, *PCAP File Format*, <https://wiki.wireshark.org/Development/LibpcapFileFormat>
- [7] cppreference.com, `std::atomic`, <https://en.cppreference.com/w/cpp/atomic/atomic>
- [8] cppreference.com, `std::shared_mutex`, [https://en.cppreference.com/w/cpp/thread/shared\\_mutex](https://en.cppreference.com/w/cpp/thread/shared_mutex)
- [9] cppreference.com, `std::optional`, <https://en.cppreference.com/w/cpp/utility/optional>